

Review article

An architectural perspective on MLOps: Structures, processes, tools, and stakeholders

Faezeh Amou Najafabadi^{ID*}, Justus Bogner^{ID}, Ilias Gerostathopoulos^{ID}, Patricia Lago^{ID}

Vrije Universiteit Amsterdam, The Netherlands

ARTICLE INFO

Keywords:

Machine learning operations
MLOps
MLOps system
MLOps architecture
MLOps process
MLOps tools
Systematic mapping study

ABSTRACT

Context: Despite the increasing adoption of Machine Learning Operations (MLOps), teams still encounter challenges in effectively applying this paradigm to their projects. While numerous MLOps tools exist, consolidated knowledge to inform architecture design is still lacking.

Objective: In response, our goal is to provide a comprehensive overview of MLOps architectures from a structural and process perspective, the tools mentioned supporting the implementation of architecture components, and the stakeholders responsible for the MLOps process.

Methods: We conduct a systematic mapping study of 93 primary studies to collect and analyze the state of the art knowledge on MLOps systems using automatic, manual, and snowballing-based search strategies. Subsequently, we use card sorting to synthesize the results.

Results: We contribute: (i) a categorization of 39 MLOps architecture components and a description of several MLOps architecture variants; (ii) a systematic map between the components and the existing MLOps tools; (iii) a description of 56 process steps for MLOps systems creation, deployment, and maintenance; and (iv) a description of MLOps stakeholders, their responsibilities, and a systematic map between the process steps and the responsible stakeholders.

Conclusion: Our results serve as an overview of the state of the art in MLOps architectures from a structural and a process perspective to support researchers and practitioners in the architecture design of their MLOps systems.

Contents

1.	Introduction	2
2.	Background and related work	3
2.1.	Background	3
2.1.1.	From DevOps to MLOps to LLMOps	3
2.1.2.	ML-enabled systems vs. MLOps systems	3
2.2.	Related work	3
3.	Research methodology	4
3.1.	Goal and research questions	5
3.2.	Research process	5
3.2.1.	Step 1: Automated initial search	5
3.2.2.	Step 2: Paper selection	6
3.2.3.	Step 3: Data extraction	6
3.2.4.	Step 4: Snowballing	6
3.2.5.	Step 5: Data synthesis	6
4.	Results	9
4.1.	MLOps architecture components and their dependencies (RQ1.1)	9
4.2.	Tools used to support or implement the components (RQ1.2)	14

* Corresponding author.

E-mail addresses: f.amou.najafabadi@vu.nl (F. Amou Najafabadi), j.bogner@vu.nl (J. Bogner), i.g.gerostathopoulos@vu.nl (I. Gerostathopoulos), p.lago@vu.nl (P. Lago).

<https://doi.org/10.1016/j.infsof.2026.108029>

Received 13 August 2025; Received in revised form 13 December 2025; Accepted 7 January 2026

Available online 24 January 2026

0950-5849/© 2026 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

4.3.	MLOps process steps and their sequence (RQ2.1)	14
4.3.1.	Perform ML experimentation	20
4.3.2.	Execute CI/CD pipeline	20
4.3.3.	Perform continuous training	21
4.3.4.	Perform ML experimentation using pre-trained models	21
4.4.	Iterative nature via feedback loops (RQ2.2)	21
4.5.	Stakeholder roles and their responsibilities (RQ2.3)	22
5.	Discussion	24
5.1.	Representation of MLOps architectures in literature	24
5.2.	Important decisions of this study	26
5.3.	Opportunities for future research	26
6.	Threats to validity	27
7.	Conclusion	27
	CRedit authorship contribution statement	28
	Declaration of competing interest	28
	Acknowledgments	28
	Data availability	28
	References	28

1. Introduction

The use of Machine Learning (ML) continues to grow in industry, and developing high-quality ML models is important to sustain it. However, when creating ML-enabled systems, another major concern of ML engineering teams is the effective deployment and maintenance of ML models in production [1]. To address this, the Machine Learning Operations (MLOps) paradigm has formed in industry [2]. Similarly to DevOps [3], MLOps comprises a set of best practices and related methods, technologies, and tools that aim to bridge the gap between the development of ML models and their deployment, operation, maintenance, and evolution. Despite the increasing adoption of MLOps [4], it is still challenging for practitioners to effectively apply the paradigm to their projects [5]. First, there is a large variety of available tools usable for MLOps, which makes it hard for practitioners to analyze and compare all the options at their disposal and to relate them to architectural components [6]. Second, while reusable design decisions for certain parts of ML-enabled systems are beginning to emerge [7], there is still a lack of consolidated MLOps architecture knowledge that could guide practitioners with a more holistic architectural vision. Designing a complete architecture and comparing common architectural variants remains challenging for industry, for ML-enabled systems in general [8], but also specifically for MLOps systems [1]. Third, MLOps evolves at a fast pace and simultaneously in different domains, which makes it difficult to discern the generalizable concepts from the domain-specific ones.

In response, the long-term goal of our research is to provide a comprehensive reference architecture [9] for MLOps that comprises (i) *structural* and *process* views, (ii) *stakeholder roles and their requirements*, as well as (iii) architecture-relevant *guidelines*. In this paper, we focus on the first aspect and provide an overview of the state of the art in MLOps from an architectural perspective, including the tools used and stakeholders involved. This paper extends an earlier work, which considered only structural aspects of MLOps architectures [10] (corresponding to the first research question in Section 3) by complementing the architecture with a process view, and the stakeholder roles within the system (corresponding to the second research question in Section 3). Additionally, the previous work was only based on 43 primary studies, while this extension synthesizes the content of 93 publications (more than twice as many). The last elements of the MLOps reference architecture, namely the requirements of the identified stakeholders and architecture-relevant guidelines, will be provided in future studies and are out of scope for this paper.

Regarding terminology, we define an **MLOps system** as a system composed of ML, non-ML, and infrastructure components that collect and process data; train and serve models; and monitor, manage, and

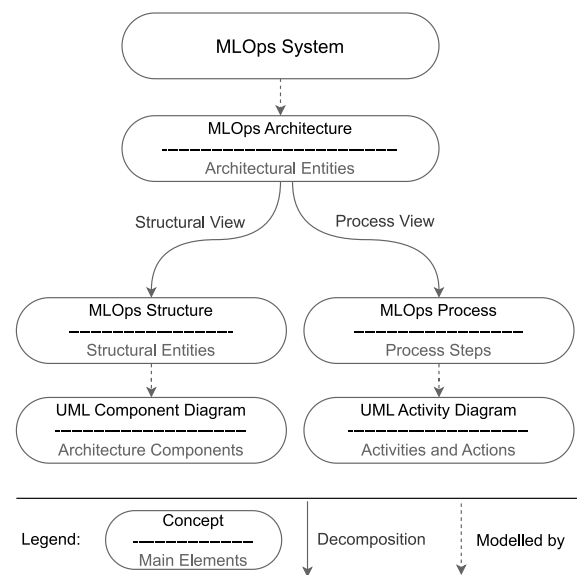


Fig. 1. The terminology used throughout this paper.

retrain inferred models. We model the architectures of such systems covering two main views, *structural* and *process*. Our synthesized model of **MLOps Architecture** is therefore composed of **MLOps Structure** and **MLOps Process** (see Fig. 1). While other views exist in software architecture, these two are the most fundamental ones to reason about a system, i.e., what parts the system has and how it behaves, which is also apparent from UML being composed of structural and behavioral diagram types.¹ Moreover, these are also views that were feasible for us to create based on figures and information provided in the average scientific paper about MLOps. For example, only very few of our included papers contained deployment-related information to create a deployment view.

In the *structural view*, we extract and analyze the **architecture components** that comprise typical architectures of MLOps systems. Moreover, we identify four main variants of such architectures based on the existing variability points, e.g., the optional presence of an online training pipeline. We then analyze the dependencies between the components and synthesize them in the form of a **UML component diagram**. To align terminology in a rather scattered domain, we provide several known aliases for each component we identify. We

¹ <https://www.uml-diagrams.org/uml-25-diagrams.html>

also extract and analyze the **tools** that are mentioned in the MLOps literature and map them to the architecture components. This provides insight into the implementation options for each component and sheds light on untapped R&D opportunities in the form of less tool-supported components.

In the *process view*, we extract and analyze the **process steps** that are undertaken throughout the design, implementation, operation, and maintenance of an MLOps system. Next, we examine the order and interdependencies of these steps and synthesize them into a **UML activity diagram**. This diagram illustrates the sequence, concurrent execution, and conditional execution of the identified steps. We also provide a mapping between the identified steps and the identified responsible **stakeholder roles**.

We achieve the above by performing a Systematic Mapping Study (SMS) of the scientific literature of MLOps, which grounds our analysis in the state of the art. As such, we complement other attempts that analyzed gray literature for MLOps best practices and architectural design decisions [7].

In summary, the contributions of this study are (i) the synthesis and categorization of 39 MLOps architecture components in 6 different groups and a description of several MLOps architecture variants; (ii) a systematic map between the identified components and the MLOps tools; (iii) a synthesis and categorization of 56 steps in 7 different phases that are taken by the stakeholders or performed by the system in the MLOps process and an overview of the MLOps process comprising the identified steps, their sequence, and feedback loops; and (iv) a thorough description of the MLOps stakeholder roles, their responsibilities, and a systematic map between the stakeholders and the process steps.

The target audience of our study are (i) researchers in software architecture for ML, who can build on our results to derive and consolidate MLOps architecture knowledge, and (ii) ML practitioners, who can use our findings to inform the architecture design of their MLOps systems.

2. Background and related work

This section presents an overview of MLOps and its evolution from DevOps, disambiguates between MLOps and ML-enabled systems, and reviews related work on MLOps.

2.1. Background

2.1.1. From DevOps to MLOps to LLMops

Bass et al. define DevOps as “a set of practices intended to reduce the time between committing a change to a system and the change being placed into normal production, while ensuring high quality” [3]. These practices are meant to enhance communication and collaboration between *development teams* responsible for developing a software system and *operations teams* responsible for deploying the system in the appropriate infrastructure and monitoring its performance in production [11–13]. Common DevOps practices such as continuous testing, integration, deployment, and delivery (together with associated tools [14]) are considered a pillar of *continuous software engineering* (CSE) [15], which has the overall goal of reducing time to market and enhancing the quality of software systems [16].

DevOps practices have to be revisited in the era of ML. With the rise of ML, more and more software systems are becoming *ML-enabled* by integrating one or more ML components to provide advanced functionality [17,18]. The alignment of development and operations in ML-enabled systems presents distinct challenges that go beyond those of traditional software systems: continuous monitoring and continuous training of ML models [2], communication between data scientists, software engineers, and DevOps engineers, and interactions between the ML and non-ML components are just a few examples. In response to this need, MLOps practices have emerged, incorporating CSE into the development and operation of ML-enabled systems [19]. The MLOps

paradigm can be therefore seen as the set of practices, guidelines, and associated tools that facilitate the collaboration among the development and operation teams and enhance the time to market and maintenance for ML-enabled systems.

Given the sharp rise in popularity and adoption of large language models (LLMs) with pioneering models such as GPT-3 and BERT, the concept of LLMops (LLM Operations) has emerged both in academic works [20–23] and in gray literature [24–30]. LLMops is “an adaptation of the MLOps methodology specifically tailored for LLMs” which “addresses particular business challenges such as cost management, engineering challenges, and requires the use of unique tools and infrastructures to effectively manage the lifecycle of LLMs” [21]. Simply put, LLMops focuses on the development, operation, and maintenance of a particular ML model type, i.e., LLMs, and the corresponding ML-enabled systems. It is a specialization of Foundation Model Operations (FMOps) focusing on language models [22].

LLMops often incorporates pre-training and fine-tuning (common to FMOps), but also in-context learning, including prompt engineering and retrieval-augmented generation [20]. At the same time, it also inherits most of the challenges and requirements of MLOps with conventional ML models [22]. By focusing on the broader MLOps concept, we simultaneously (i) cover the highly important and emerging topic of LLMops architectures and (ii) investigate whether LLMops architectures can be considered a specialization of MLOps architectures or have enough unique characteristics to render this view impractical. We revisit this point in the Discussion (Section 5).

2.1.2. ML-enabled systems vs. MLOps systems

For the purpose of this work, we define an MLOps system as a system that includes (i) pipelines for data processing, ML training, and ML serving (supported by different tools and infrastructure) and (ii) mechanisms for ML model monitoring and model retraining/repeated fine-tuning. Similar to an ML-enabled system, we view an MLOps system as being composed of ML, non-ML, and infrastructure components. However, we differentiate between the two **based on the focus that is placed on important components and stakeholders**. In ML-enabled systems, greater emphasis is placed on business-related (non-ML) components, such as *domain-related services*, *UI components*, and *authentication and authorization services*, i.e., it is more important how the trained ML models are used during inference and how they interact with potential other non-ML components. Conversely, in MLOps systems, the focus shifts towards infrastructure components like *ML training pipelines*, *CI/CD pipelines*, and *model monitoring components*, i.e., it is more important how the ML models are continuously created, evaluated, integrated, deployed, and monitored. In addition to these software-oriented elements, MLOps also encompasses hardware management concerns—most notably, selecting appropriate compute resources and balancing workloads across available hardware to ensure efficient and reliable operation. Moreover, stakeholders in an ML-enabled system focus on either creating or using the system. In contrast, in an MLOps system, stakeholders such as data scientists simultaneously contribute to system development and continuously use the system to further enhance it. Consequently, in MLOps systems, the form of interaction between the stakeholders and the system is more integrated and iterative. Fig. 2 provides an overview of the concepts used for ML-enabled systems and MLOps systems.

2.2. Related work

MLOps definitions, practices, and guidelines have been the subject of numerous secondary studies, including scoping reviews, Systematic Literature Reviews (SLRs), and Multivocal Literature Reviews (MLRs).

Many secondary studies on this topic aim at **clarifying the definition of MLOps**. Mboweni et al. [31] state that there is still no official standard definition for MLOps. Based on their systematic review to disambiguate the definition of MLOps in the literature, they

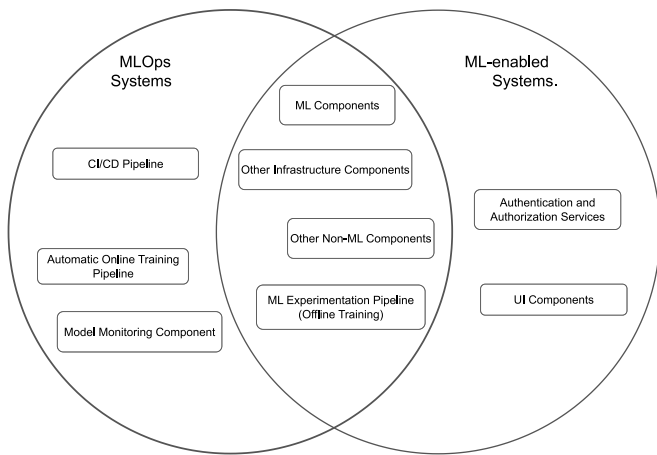


Fig. 2. The differences and intersection between MLOps and ML-enabled systems, based on the primary focus areas of each. Note that this is not a perfect representation; rather, it highlights only the concepts related to our work and represents the main – though not exclusive – focus of each system type.

claim that they did not find evidence of a common understanding among scholars and experts on how MLOps should be implemented and institutionalized across the industry to create a common vision. Lima et al. [32] systematically reviewed 30 papers aiming to derive practices, standards, roles, maturity models, challenges, and tools for MLOps. Based on the addressed challenges and assessment of models, they draw the conclusion that “research on MLOps is still in its initial stages.” With the rise and rapid interest in LLMs, some papers also try to provide a definition and define the characteristics of LLMops. Diaz-De-Arcaya et al. [21] conduct a multivocal literature review and define LLMops as “an adaptation of the MLOps methodology specifically tailored for LLMs” and analyze the stages that are performed in LLMops. They emphasize that considering the limited literature on LLMops, the derived definition and view over the LLMops stages may be limited.

Some literature reviews, in turn, provide methodologies for **effectively approaching MLOps projects**. Testi et al. [33] provide a taxonomy of the current approaches and propose a methodology for addressing MLOps projects. Kolltveit and Li [34] specifically focus on the operationalization of ML models with regard to tools and infrastructure that are deployed in different stages of MLOps workflows. Recupito et al. [35] take a different perspective and provide an overview of the most common tools and their characteristics that support the creation of MLOps pipelines, without a clear mapping to components. Vadde and Munagandla [36] propose a framework for building and deploying robust, scalable, and maintainable ML pipelines. To provide their framework, the authors explore different techniques for implementing the components of the MLOps system. In a recent study, Warnett et al. [37] provide a rating scheme for MLOps systems to understand their support for automation-related architectural design decisions. By analyzing several MLOps systems, architectural descriptions, and MLOps reference architectures, the authors model 22 MLOps models and define objective, technology-agnostic metrics to measure conformance of the models to architectural design decision options. Based on the modeling and the metrics, the authors demonstrate that “simple yet effective metrics provide reliable insights into crucial system aspects.”

Several papers on the **architecture of ML-enabled systems** are also closely related to our study. For example, Warnett and Zdun conducted two studies in which they used practitioner gray literature to synthesize architectural design decisions (ADDs) for ML workflows [38] and ML deployment [7], with several of their sources being blog posts about

MLOps. As a result, several of their ADDs are related to architecture components that we synthesize in our study. However, they do not combine this knowledge into a holistic architecture and also do not cover several parts of MLOps, such as inference and monitoring. In a controlled experiment, Warnett and Zdun [39] also compared the understandability of informal textual and graphical MLOps architecture representations with semiformal MLOps architecture diagrams. They conclude that the understandability of MLOps architecture descriptions is significantly better with supplementary semiformal architecture diagrams. Kumara et al. [40] strive towards a reference architecture of MLOps by eliciting requirements and components from the gray literature. In their preprint, they provide a layered architecture that focuses on the requirements that the MLOps environment needs to provide. Erdogan et al. [41] provide a process model for AI-enabled software development. In the 14 reviewed papers in their SLR, the authors find “no example to propose an end-to-end development life cycle for AI software”. Similar to our synthesis, they provide main concerns and supporting activities in their process model, which we refer to as “core” and “add-on” steps in our process view. However, the provided process model by the authors models a general AI-enabled system and is more coarse-grained as opposed to our process view of the MLOps systems architecture, which is composed of many more fine-grained steps. Lastly, Wozniak et al. [42] provide the results of a systematic literature review where they report on MLOps components, process steps, and tools, and map the tools to architectural components. They also provide a reference architecture in the form of an informal representation of the components and the most common tool for implementing each component. However, what they represent as a reference architecture is an informal representation of selected components, which can be complemented by other architectural views and practices.

In ML-enabled systems, the task of working with ML models is generally assigned to the **role of the data scientist**. This term was first introduced by DJ Patil and Jeff Hammerbacher in 2008 to define their roles at LinkedIn [43]. Kim et al. [44] provide trends about data science roles within the Software Engineering (SE) context. The authors define the responsibility of data scientists to transform data into insights that guide team actions and explain that data scientist is a new emerging role in software teams — most participants in their survey were initially hired in other engineering roles and have taken the new responsibilities as part of data science later on. Saltz and Grady [45] emphasize the ambiguity of the data science roles by conducting a case study on five different cases and show that barely two roles have the same title or are described the same across the examined cases. They also evaluate the identified roles by searching for them on a technology-focused job website. Surprisingly, they find that some identified roles do not appear in any job description.

Despite the existing literature regarding defining and disambiguating MLOps, we identify a lack of a descriptive model-based conceptualization of what MLOps encompasses as a holistic system. Such a model can contribute to a unified common understanding of MLOps from a system perspective, which, as shown above, is one of the concerns in the field. Our own study complements the above studies by using scientific literature to synthesize MLOps system components, supporting tools to implement them, MLOps process steps, and stakeholders responsible for or affected by each step, unified into one model of the MLOps systems described via structural and process perspectives. In addition, unlike existing works, we also synthesize and discuss several architecture variants and a rigorous end-to-end process description. MLOps researchers and practitioners can use the proposed architecture variants (of the structural and process views) to familiarize themselves with common architectural elements and to identify a suitable variant for their requirements.

3. Research methodology

In this section, we describe the research goals and the process of our study. We design and follow a rigorous protocol by following established guidelines for systematic secondary studies [46].

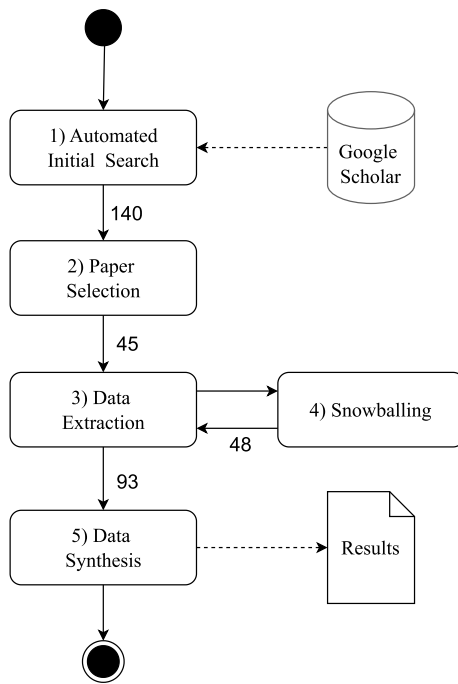


Fig. 3. Overview of the research process. Numbers on transitions refer to papers considered as input to each step.

3.1. Goal and research questions

The main goal of this SMS is to **identify, classify, and analyze the architectures of existing MLOps systems described in scientific literature**. In particular, as illustrated in Fig. 1, we focus on their structural and process views, two architectural blueprints that will serve as key elements of our envisioned comprehensive reference architecture. In the structural view, we focus on the individual components, their dependencies, responsibilities, and the tools that support them. In the process view, the focus is on the identified steps, their sequence, including feedback loops, and the responsible stakeholders in each.

To approach this goal, we formulate the following two Research Questions (RQs), each consisting of research sub-questions:

RQ1: How are MLOps structures described in the scientific literature?

The main outcome of this research question is the state of the art of the structural view of the architecture of an MLOps system. In this regard, we aim to identify how MLOps systems are conceptualized across different application domains.

RQ1.1: Which are the different components and their dependencies? This research sub-question helps us categorize the various components within an MLOps architecture and comprehend how different researchers and practitioners interconnect these components.

RQ1.2: Which tools are used to support or implement the identified components? The primary focus of this research sub-question is to establish a concise mapping between the different components in the architecture and the employed tools for these components. Considering the myriad of available tools that claim to be useful for different components of MLOps systems, we aim to find out if and which tools are the most recommended for specific MLOps components.

RQ2: How are MLOps processes described in the scientific literature?

This research question seeks to identify how MLOps processes are characterized and defined in the state of the art. The expected output of this research question is an ordered list of the different steps in an MLOps process.

RQ2.1: What are the different steps and their sequence? With this research sub-question, we plan to identify the proposed individual steps of a comprehensive MLOps process, the order in which they are performed, and also their similarities and differences in different papers.

RQ2.2: How are the iterative aspects of the process conveyed? Given the inherently iterative nature of the MLOps process, we aim to analyze how iteration is described and represented in the scientific literature, e.g., via explicitly modeled feedback loops.

RQ2.3: Which MLOps stakeholder roles support each of the identified steps? This research sub-question assists us in providing a clear mapping between the different steps in the process of an MLOps workflow and the different stakeholder roles that are reported as responsible for them.

3.2. Research process

We visualize our research process in Fig. 3 and describe the different steps in this section. Essentially, we first obtain an initial set of papers via an automated title-focused search on Google Scholar, then filter the results to obtain a starting set of primary studies. After extracting data from the first set of primary studies, we augment this set via bi-directional one-step snowballing [47] and extract data from the new set of primary studies. Using Google Scholar as a meta-engine allows us to avoid bias towards specific publishers [48]. Finally, the extracted data is used in a rigorous synthesis process. It is worth mentioning that this process, from the automated initial search to data synthesis, is repeated twice. We performed the first round of query search in September 2023 and published the results as a conference paper [10]. After that, we expanded our research and also included LLMops architectures, when we performed the second round of our research in February 2025, for which the results are presented in this journal paper.

3.2.1. Step 1: Automated initial search

By extracting relevant search terms from MLOps literature known to us and adding synonyms, we iteratively construct and test several search strings. As a result, we finally arrive at the following queries to search in the titles of the available literature:

allintitle: (MLOps OR "machine learning operations") AND (model OR models OR pipeline OR pipelines OR architecture OR architectures OR architecting OR workflow OR workflows OR process OR processes OR design OR designs OR designing OR pattern OR patterns OR style OR styles)

allintitle: (LLMOps OR "large language model operations") AND (model OR models OR pipeline OR pipelines OR architecture OR architectures OR architecting OR workflow OR workflows OR process OR processes OR design OR designs OR designing OR pattern OR patterns OR style OR styles)

The advantages of these final title-focused variants are the manageable number of results (140 resulting sources) and the low number of false positives. Additionally, potential limitations regarding the extensiveness of the results are compensated for by snowballing. The automated queries are executed via Google Scholar at two points in time: first in September 2023 (only the first query), and then in February 2025 (to incorporate newer papers and expand our results to include LLMops as well). These lists are completed at a later stage via bi-directional snowballing as described in Step 4.

Table 1
Data extraction framework.

RQ	Data item	Notes
Generic	Application domain	The respective domain the architecture is proposed for
Generic	Author affiliations	Origin of the paper: academia, industry, or collaboration
RQ1, RQ2	Architecture figures	The list of relevant figures and their types (values can be structure, process, or combined)
RQ1.1	Structural entities	The list of structural entities and their relationship
RQ1.2	Tools	The list of tools that are used or suggested in the paper
RQ1.2	Tool-entity mapping	The list of tools mapped to the structural entities
RQ2.1, RQ2.2	Process steps	The list of steps and their relationship including feedback loops.
RQ2.3	Stakeholders	The list of MLOps stakeholders mentioned in the paper
RQ2.3	Stakeholder-step mapping	The mentioned stakeholders mapped to the process steps

3.2.2. Step 2: Paper selection

The following inclusion (I) and exclusion (E) criteria are used during the selection of primary studies.

Inclusion criteria:

- I1 The paper describes the architecture of an MLOps system or part thereof.
- I2 The described MLOps architecture is an original contribution of the paper, not simply cited related work.

Exclusion criteria:

- E1 The paper is not written in English.
- E2 The paper is a shorter or earlier version of a paper that is already included.
- E3 The paper is a pure secondary or tertiary study.
- E4 The full text of the paper is not available.
- E5 The paper is not peer-reviewed and published.

To be included in the list of primary studies, a paper has to fulfill all inclusion and no exclusion criteria. To arrive at the above selection criteria, we conduct a selection pilot in which two reviewers independently review five papers and discuss their selection strategy in a consensus meeting. After fine-tuning the selection process, it is applied to all potentially relevant papers. Each paper is independently evaluated by two researchers for inclusion, with a consensus being necessary for final inclusion. In the first and second rounds of the automated search, applying the selection criteria results in 18 and 27 papers that form our starting set used in the first round of data extraction and in snowballing.

3.2.3. Step 3: Data extraction

In this step, we systematically analyze the primary studies and extract data related to the RQs. To refine our data extraction strategy, we first conduct a data extraction pilot on five randomly chosen papers. Data from each paper is extracted by two authors independently and discussed in a consensus meeting, leading to the final data extraction framework depicted in Table 1. For both RQ1 and RQ2, we extract the item “Architecture figures”. For the research sub-questions of RQ1, we extract the items “Structural entities”, “Tools”, and “Tool-entity mapping”. For the research sub-questions of RQ2, we extract the items “Process steps”, “Stakeholders”, and “Stakeholder-step mapping”. The “Application domain” and the “Author affiliations” are extracted as generic information to use for further analysis. From this point on, each of the remaining papers is assigned to two authors for data extraction. The extracted data is validated through bilateral discussions and consensus between the authors.

3.2.4. Step 4: Snowballing

We apply backward and forward snowballing to enrich the results obtained via automated search, as suggested by Wohlin et al. [47].

During this step, all papers that either cite or are cited by a paper from the starting set are examined for inclusion in the final set of primary studies. We apply both the same selection criteria and data extraction (steps 2 and 3) as for the initial set of papers and the same process: each paper is examined independently by two researchers and a consensus needs to be reached for including it, then the data from each paper is extracted by two researchers and the results are discussed. After conducting a first round of bidirectional snowballing, we add more papers to our set of primary studies. We limit the snowballing process both times to single rounds; this decision stemmed from the observation that subsequent data extraction yielded only very few additional components, tools, steps, and stakeholder roles compared to the initial seed collection. In the end, we synthesize data from 93 selected papers in both iterations of the research process. Table 2 provides a full list of primary studies that we reviewed.

3.2.5. Step 5: Data synthesis

In this phase, we harmonize and classify the extracted data per parameter (structural entities, tools, process steps, stakeholders). To achieve this synthesis, we use *card-sorting*, a lightweight, collaborative, qualitative analysis technique [138].

In particular, we use *hybrid card-sorting* [139], a combination of open card-sorting (where categories emerge from the data) and closed card-sorting (categories are defined beforehand based on existing taxonomies). We define the initial set of categories based on our background knowledge of MLOps and software architecture, and iteratively refine and enrich this set. We conduct two runs of card-sorting, focusing separately on the structural view and on the process view. In the first run, we conduct the card-sorting in three phases: in the preparation phase, we print all the extracted components on cards; in the execution phase, we sort the cards into meaningful categories and groups and name them; in the analysis phase, we identify the relationships between the identified components and categories, and we label the components as *Core* or *Add-on*. We also disambiguate and group together the extracted tools and derive their mapping to the synthesized architecture components. In the second run, we perform the same process, this time for the steps of the process view, and for the related stakeholders.

For labeling a structural entity or process step as *Core* or *Add-on* (as described and shown in Section 4), we rely on a two-step process. First, we identify *Core* components based on the description of their responsibilities in the primary studies. We deliberately did not base this decision exclusively on the number of occurrences of each entity in the primary studies, since it is very difficult to provide a clear-cut criterion based on this number for considering an entity as *Core* or not. Second, we augment this list with any other component that we consider necessary to maintain a consistent synthesized model as a result of our synthesis. The second step is needed since several architectural entities may not be emphasized in the literature but are still mandatory for a consistent view of an MLOps system, meaning that removing them would yield dangling components or unreachable steps. Examples of such entities are *ML Model Deployer* for the structural view and *Select Data* for the process view, which are not explicitly mentioned in many

Table 2

List of primary studies.

ID	Title	Author(s)	Ref
S1	MLOps Pipeline Development: The OSSARA Use Case	Sergio Moreschini, David Hästbacka, Davide Taibi	[49]
S2	Model-Driven Approach to Blockchain-Enabled MLOps	Nenad Petrović	[50]
S3	Towards MLOps: A case study of ML pipeline platform	Yue Zhou, Yue Yu, Bo Ding	[51]
S4	Towards MLOps in Mobile Development with a Plug-in Architecture for Data Analytics	Rustem Dautov, Erik Johannes Husom, Fotis Gonidis	[52]
S5	Model provenance management in MLOps Pipeline	Songzhu Mei, Cong Liu, Qinglin Wang, Huayou Su	[53]
S6	An Efficient Microservices Architecture for MLOps	Seol Roh, Ki-Moon Jeong, Hye-Young Cho, Eui-Nam Huh	[54]
S7	Machine Learning Operations (MLOps): Overview, Definition, and Architecture	Dominik Kreuzberger, Niklas Kühl, Sebastian Hirschl	[1]
S8	Docker and Kubernetes Pipeline for DevOps Software Defect Prediction with MLOps Approach	Rizal Broer Bahaweres, Aldi Zulfikar, Irman Hermadi, Arif Imam Suroso, Yandra Arkeman	[55]
S9	A digital twin architecture model applied with MLOps techniques to improve short-term energy consumption prediction	Tiago Yukio Fujii, Victor Takashi Hayashi, Reginaldo Arakaki, Wilson Vicente Ruggiero, Romeo Bulla Jr, Fabio Hirotsugu Hayashi, Khalil Ahmad Khalil	[56]
S10	MLOps approach in the cloud-native data pipeline design	István Póloskei	[57]
S11	SensiX++: Bringing mlops and multi-tenant model serving to sensory edge devices	Chulhong Min, Akhil Mathur, Utku Gunay Acer, Alessandro Montanari, Fahim Kawsar	[58]
S12	MILKI-PSY Cloud: MLOps-based Multimodal Sensor Stream Processing Pipeline for Learning Analytics in Psychomotor Education	Ralf Klamma, Michal Piotr Slupczynski	[59]
S13	Accelerating the Machine Learning Model Deployment using MLOps	Mandepudi Nobel Chowdary, Bussa Sankeerth, Chennupati Kumar Chowdary, Manu Gupta	[60]
S14	Jenkins Pipelines: A Novel Approach to Machine Learning Operations (MLOps)	D. R.Niranjana, Mohana	[61]
S15	K2E: Building MLOps Environments for Governing Data and Models Catalogues while Tracking Versions	Gorka Zárate, Raúl Miñón, Josu Díaz-de-Arcaya, Ana I. Torre-Bastida	[62]
S16	Towards a safe MLOps Process for the Continuous Development and Safety Assurance of ML-based Systems in the Railway Domain	Marc Zeller, Thomas Waschulzik, Reiner Schmid, Claus Bahlmann	[63]
S17	Development of MLOps Platform Based on Power Source Analysis for Considering Manufacturing Environment Changes in Real-Time Processes	Ji-hyun Cha, Heung-gyun Jeong, Seung-woo Han, Dong-chul Kim, Jung-hun Oh, Seok-hee Hwang, Byeong-ju Park	[64]
S18	Speeding Up the Machine-learning Process with MLOps and Creating a Mechanism to Continuously Provide Service Value	Yamaguchi Ei	[65]
S19	MLOps for evolvable AI intensive software systems	Sergio Moreschini, Francesco Lomio, David Hästbacka, Davide Taibi	[66]
S20	Edge MLOps: An Automation Framework for AIoT Applications	Emmanuel Raj, David Buffoni, Magnus Westerlund, Kimmo Ahola	[67]
S21	Demystifying MLOps and Presenting a Recipe for the Selection of Open-Source Tools	Philipp Ruf, Manav Madan, Christoph Reich, Djaffar Ould-Abdeslam	[68]
S22	From DevOps to MLOps: Overview and Application to Electricity Market Forecasting	Rakshith Subramanya, Seppo Sierla, Valeriy Vyatkin	[19]
S23	A reference architecture for the operationalization of machine learning models in manufacturing	Tim Raffina, Tobias Reichensteina, Jonas Wernera, Alexander K'uhla, Jörg Franke	[69]
S24	Applying DevOps Practices of Continuous Automation for Machine Learning	Ioannis Karamitsos, Saeed Albarhami, Charalampos Apostolopoulos	[70]
S25	Building A Platform for Machine Learning Operations from Open Source Frameworks	Yan Liu, Zhijing Ling, Boyu Huo, Boqian Wang, Tianen Chen, Esma Mouine	[71]
S26	Tiny-MLOps: a framework for orchestrating ML applications at the far edge of IoT systems	Mattia Antonini, Miguel Pincheira, Massimo Vecchio, Fabio Antonelli	[72]
S27	Edge Impulse: An MLOps Platform for Tiny Machine Learning	Janapa Reddi, Vijay and Elium, Alexander and Hymel, Shawn and Tischler, David and Situnayake, Daniel and Ward, Carl and Moreau, Louis and Plunkett, Jenny and Kelcey, Matthew and Baaijens, Mathijs and others	[73]
S28	Challenges and Experiences with MLOps for Performance Diagnostics in Hybrid-Cloud Enterprise Software Deployments	Amitabha Banerjee, Chien-Chia Chen, Chien-Chun Hung, Xiaobo Huang, Yifan Wang, Razvan Chevesaran	[74]
S29	StreamMLOps: Operationalizing Online Learning for Big Data Streaming & Real-Time Applications	M. Barry and J. Montiel and A. Bifet and S. Wadkar and N. Manchev and M. Halford and R. Chiky and S. Jaouhari and K. B. Shakman and J. Fehaily and F. Le Deit and V. Tran and E. Guerizec	[75]
S30	Smart Farming Monitoring Using ML and MLOps	Yaganteeswarudu Akkem, Saroj Kumar Biswas, Aruna Varanasi	[76]
S31	Autonomously Adaptive Machine Learning Systems: Experimentation-Driven Open-Source Pipeline	Yumo Luo, Mikko Raatikainen, Jukka K. Nurminen	[77]
S32	Model Operationalization at Edge Devices	Shikhar Kwatra, Utpal Mangla, Mudhakar Srivatsa	[78]
S33	MLOps for Scarce Image Data: A Use Case in Microscopic Image Analysis	Angelo Yamachui Sitcheu, Nils Friederich, Simon Baeuerle, Oliver Neumann, Markus Reischl, Ralf Mikut	[79]
S34	Continual-Learning-as-a-Service (CLaaS): On-Demand Efficient Adaptation of Predictive Models	Rudy Semola, Vincenzo Lomonaco, Davide Bacciu	[80]
S35	Conceptualizing the Secure Machine Learning Operations (SecMLOps) Paradigm	Xinrui Zhang, Jason Jaskolka	[81]
S36	Fundamental requirements of a machine learning operations platform for industrial metal additive manufacturing	Mutahar Safdar, Padma Polash Paul, Guy Lamouche, Gentry Wood, Max Zimmermann, Florian Hannesen, Christophe Bescond, Priti Wanjara, Yaoyao Fiona Zhao	[82]

(continued on next page)

Table 2 (continued).

ID	Title	Author(s)	Ref
S37	Is MLOps different in Industry 4.0? General and Specific Challenges	Leonhard Faubel, Klaus Schmid and Holger Eichelberger	[83]
S38	Tailoring MLOps Techniques for Industry 5.0 Needs	Csaba Hegedűs, Pál Varga	[84]
S39	MLOps for Enhancing the Accuracy of Machine Learning Models using DevOps, Continuous Integration, and Continuous Deployment	Mediseti Yashwanth Sai Krishna, Suresh Kumar Gawre	[85]
S40	Aspects of Module Placement in Machine Learning Operations for Cyber Physical Systems	Philipp Ruf, Christoph Reich, Djaffar Ould-Abdeslam	[86]
S41	MLOps Scaling Machine Learning Lifecycle in an Industrial Setting	Yizhen Zhao, Adam S. Z. Belloum, Gonçalo Maia da Costa, Zhiming Zhao	[87]
S42	Deception and Continuous Training Approach for Web Attack Detection using Cyber Traps and MLOps	Van-Hau Pham, Hoang Khoa Nghi, Huu Quyen Nguyen, Duy The Phan	[88]
S43	LinkEdge: Open-sourced MLOps Integration with IoT Edge	Savidu Dias, Ella Peltonen	[89]
S44	A Machine Learning Operations (MLOps) Monitoring Model Using BI-LSTM and SARSA Algorithms.	Elgama, Zeinab Shoieb; Elfangary, Laila; Fahmy, Hanan;	[90]
S45	Machine Learning Operations Applied to Development and Model Provisioning	Méndez, Óscar A; Camargo, Jorge; Florez, Hector;	[91]
S46	End-to-End Automation of ML Model Lifecycle Management using Machine Learning Operations Platforms	Hsu, Chung-Chian; Chen, Pin-Han; Wu, I-Zhen;	[92]
S47	An Architecture to Integrate Digital Twins and Machine Learning Operations	van Bruggen, Arno H; Kruger, Karel; Basson, Anton H; Grobler, Jacomine;	[93]
S48	Enhanced FIWARE-Based Architecture for Cyberphysical Systems With Tiny Machine Learning and Machine Learning Operations: A Case Study on Urban Mobility Systems	Conde, Javier; Munoz-Arcentales, Andrés; Alonso, Álvaro; Salvachúa, Joaquín; Huecas, Gabriel;	[94]
S49	Big Data-driven MLOps workflow for annual high-resolution land cover classification models	Burgueño-Romero, Antonio M; Barba-González, Cristóbal; Aldana-Montes, José F;	[95]
S50	Automating ML Models Using MLOPS	Sirisha, N; Kiran, Ajmeera; Arshad, Mohammad; Mounika, M;	[96]
S51	Dynamic Tracking, MLOps, and Workflow Integration: Enabling Transparent Reproducibility in Machine Learning	Safri, Hamza; Papadimitriou, George; Deelman, Ewa;	[97]
S52	An MLOps Approach for Deploying Machine Learning Models in Healthcare Systems	Mallardi, Giulio; Calefato, Fabio; Quaranta, Luigi; Lanubile, Filippo;	[98]
S53	Fostering Agricultural Transformation through AI: An Open-Source AI Architecture Exploiting the MLOps Paradigm	Cob-Parro, Antonio Carlos; Lalangui, Yerhard; Lazcano, Raquel;	[99]
S54	A MLOps Architecture for XAI in Industrial Applications	Faubel, Leonhard; Woudsma, Thomas; Methnani, Leila; Ghezljhemeidan, Amir Ghorbani; Buelow, Fabian; Schmid, Klaus; Van Driel, Willem D; Kloepper, Benjamin; Theodorou, Andreas; Nosratinia, Mohsen;	[100]
S55	Machine Learning Models Monitoring in MLOps Context: Metrics and Tools.	Bodor, Anas; Hnida, Meriem; Daoudi, Najima;	[101]
S56	Efficient MLOps Pipeline for Transfer Learning and Reuse of Pre-Trained ML Models	Kumar, Vimal; Ghosh, Debjani; Srivastava, Shivom;	[102]
S57	Framework for Automated Machine Learning Workflows: Building End-to-End MLOps Tools for Scalable Systems on AWS	Jana, AK;	[103]
S58	Towards Architecting Sustainable MLOps: A Self-Adaptation Approach	Bhatt, Hiya; Arun, Shrikara; Kakran, Adyansh; Vaidhyanathan, Karthik;	[104]
S59	Towards an open-source MLOps architecture	Burgueño-Romero, Antonio M; Benítez-Hidalgo, Antonio; Barba-González, Cristóbal; Aldana-Montes, José F;	[105]
S60	From Development to Deployment: An Approach to MLOps Monitoring for Machine Learning Model Operationalization	Bodor, Anas; Hnida, Meriem; Najima, Daoudi;	[106]
S61	Unlocking the Power of Data in Telecom: Building an Effective MLOps Infrastructure for Model Deployment	Nia, Amirhossein Hossein; Kaleibar, Farhoud Jafari; Feizi, Fatemehzahra; Rahimi, Fatemeh; Kashfi, Houman;	[107]
S62	Automated Credit Card Default Prediction Using MLOps on AWS: From Pipeline Development to Real-Time Deployment	Tripathi, Rajnee;	[108]
S63	Multi-Level XAI-Driven MLOps Pipeline for the Adjustment of Fruit and Vegetable Classifiers	Iriarte, Francisco J; Ortiz, Miguel E; Unzueta, Luis; Martínez, Javier; Zaldivar, Javier; Arganda-Carreras, Ignacio;	[109]
S64	NSSF Function in 6G Networks Based on MLOps Deployment Model	Da Silva, Douglas Chagas; de Sousa, Marco Antonio Firmino; De Oliveira, Wagner; Barbosa, Akio; Bressan, Graça; Silveira, Regina Melo; Henrique, Paulo Sergio Rufino; Prasad, Ramjee;	[110]
S65	A Formal Model for Integrating Consent Management into MLOps	Peyrone, Neda; Wichadakul, Duangdao;	[111]
S66	Predictive Maintenance for Wind Turbine Bearings: An MLOps Approach with the DIAFS Machine Learning Model	Shakhovska, Nataliya; Campos, Jaime;	[112]
S67	Enterprise LLMops: Advancing Large Language Models Operations Practice	Shan, Richard; Shan, Tony;	[23]
S68	Integration of web scraping, fine-tuning, and data enrichment in a continuous monitoring context via large language model operations.	Bodor, Anas; Hnida, Meriem; Daoudi, Najima;	[113]
S69	Brain Tumor Detection using MLOps and Hybrid Multi-Cloud	Archit Jain; Adarsh Malviya; Disha Bajaj; Revati Bhavsar; Amit Savyanavar	[114]
S70	A Conceptual Architecture for AI in Supply Chain Risk Management	Souma (Kanti) Paul; Sadia Riaz; Suchismita Das	[115]

(continued on next page)

Table 2 (continued).

ID	Title	Author(s)	Ref
S71	An Agile Software Development Life Cycle Model for Machine Learning Application Development	Romesh Ranawana; Asoka S. Karunananda	[116]
S72	MLOps in freight rail operations	Juan Pineda-Jaramillo, Francesco Viti	[117]
S73	Towards Regulatory-Compliant MLOps: Oravizio's Journey from a Machine Learning Experiment to a Deployed Certified Medical Product	Tuomas Granlund, Vlad Stirbu & Tommi Mikkonen	[118]
S74	The Use of eXplainable Artificial Intelligence and Machine Learning Operation Principles to Support the Continuous Development of Machine Learning-Based Solutions in Fault Detection and Identification	Tuan-Anh Tran, Tamás Ruppert, János Abonyi	[119]
S75	Machine Learning Pipeline For Anomaly Detection In Next Generation Networks	Akash Gopikrishnan; Alqama Rao; Arun Prakash; Varun Gowtham; Florian Schreiner; Marius Corici	[120]
S76	DevoX: Deep Voice Detection MLOps Framework for Crime Prevention in the Core-Edge Cloud	Yuri Seo; Hyeon-ki Jo; Seol Roh; Teh-Jen Sun; Seung-Woo Jeong; HakHo Kim	[121]
S77	No-code MLOps Platform for Data Annotation	Hyeongbok Kim; Beomjin Kim; Wenxin Lu; Lu Li	[122]
S78	MLOps: A Multiple Case Study in Industry 4.0	Leonhard Faubel; Klaus Schmid	[123]
S79	MLOps as Enabler of Trustworthy AI	Yann Billeter; Philipp Denzel; Ricardo Chavarriaga; Oliver Forster; Frank-Peter Schilling; Stefan Brunner	[124]
S80	MLOps for Developing Machine-Learning-Enhanced Automotive Applications: Experience at the Bosch Cross-Domain Computing Solutions Division	A. L. Ferreira and J. M. Fernandes	[125]
S81	A Machine Learning Operations Platform for Streamlined Model Serving in Industry 5.0	Colombi, Lorenzo and Gilli, Alessandro and Dahdal, Simon and Boleac, Ion and Tortonesi, Mauro and Stefanelli, Cesare and Vignoli, Massimiliano	[126]
S82	LLMOps: Definitions, Framework and Best Practices	Sinha, Megha and Menon, Sreekanth and Sagar, Ram	[22]
S83	Quality Assurance in MLOps Setting: An Industrial Perspective	Ayan Chatterjee, Bestoun S Ahmed, Erik Hallin, Anton Engman	[127]
S84	Role of Regulatory Sandboxes and MLOps for AI-Enabled Public Sector Services	Ana Paula Gonzalez Torres, Nitin Sawhney	[128]
S85	The machine learning life cycle and the cloud: implications for drug discovery	Ola Spjuth, Jens Frid, Andreas Hellander	[129]
S86	An Automatic Artificial Intelligence Training Platform Based on Kubernetes	Chaoyu Wu, E. Haihong, Meina Song	[130]
S87	Building Continuous Integration Services for Machine Learning	Bojan Karlaš, Matteo Interlandi, Cedric Renggli, Wentao Wu, Ce Zhang, Deepak Mukunthu Iyappan Babu, Jordan Edwards, Chris Lauren, Andy Xu, Markus Weimer	[131]
S88	Kidney Tumor Detection Using MLflow, DVC and Deep Learning	G, Usha M and S, Shreya M and S, Supreeth and G, Shruthi and Pruthviraja, Dayananda and Chavan, Pundalik	[132]
S89	Deployment and Serving ML Models Using Kubeflow and KfServing	R. J. Priyankasingh and H. Y. Vani	[133]
S90	Secure and Scalable LLM-Based Recommendation Systems: An MLOps and Security by Design	Saputra, Adi and Suryani, Erma and Rakhmawati, Nur Aini	[134]
S91	A Framework for Adopting Machine Learning in the Clinical Domain	Esther Meng-Yoke Tan, Fan Liu, Yuen Kwan Chia	[135]
S92	A Design Framework for Cloud-HPC Integration in MLOps	Siilasjoki, Niila, Mikko Raatikainen, and Jukka K. Nurminen	[136]
S93	Cloud computing design patterns for MLOps: applications to virtual power plants	Rakshith Subramanya, Paula R'ais'anen, Seppo Sierla, and Valeriy Vyatkin	[137]

architectures yet fundamental. Lastly, we conduct a final control review to ensure a consistent synthesis between the identified architecture components and the corresponding process steps.

The final output is a general synthesis of the existing MLOps architectures from structural and process perspectives (represented in a UML component diagram and a UML activity diagram, respectively), along with mappings between tools and components, and steps and stakeholders. Lastly, we synthesize the information about the mentioned domains and also create groups of the domains for which the MLOps architectures are proposed.

4. Results

The results of our study are based on extracted and synthesized data from 93 papers published between 2020 and the beginning of 2025. This time frame aligns with the emergence and maturation of the MLOps paradigm, which is placed in early 2019 [4].

We observe an increasing interest in the publications by year on this topic, with the highest number of publications in 2024 (27/93 papers).

Generally, we observe that less than half of the papers (39/93) propose domain-agnostic architectures, and the rest propose the architecture for a specific case in a domain. Among the domain-specific

reviewed architectures, the top three domains include *industry and manufacturing* (12 papers), *healthcare and sports science* (9 papers), and *edge computing* (8 papers). This is presented in Fig. 4. Moreover, as depicted in Fig. 5, 58/93 papers are authored by academic researchers, 12/93 by industry practitioners, and 23/93 as a collaboration between the two. This is a testament to the strong interest of *both* academia and industry in providing architecture guidance for MLOps systems. In the remainder of this section, we present the study results organized according to the research questions and sub-questions.

4.1. MLOps architecture components and their dependencies (RQ1.1)

In total, we synthesized 39 unique architecture components by systematically going through all the architecture figures and descriptions contained in our 93 primary studies. These components are domain-agnostic, i.e., they are not tied to a specific application domain. We also identified domain-specific components such as *IoT Sensors* (IoT domain) or *User Feedback Collector* (psychomotor learning domain), but excluded them for the sake of general applicability. We group the 39 identified architecture components into 6 categories:

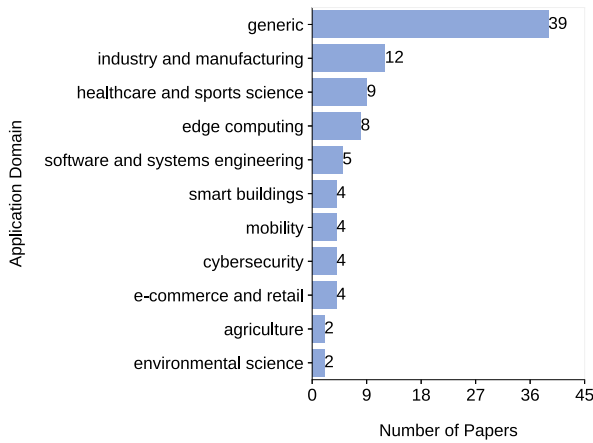


Fig. 4. Number of selected publications per application domain.

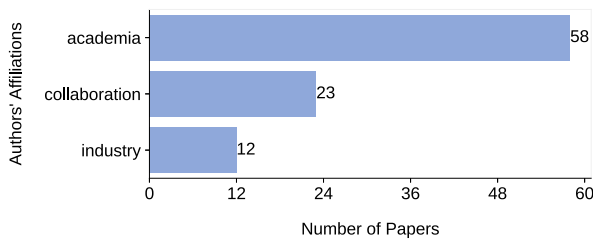


Fig. 5. Number of selected publications per authors' affiliations.

- *Data Curation* entails components responsible for gathering and processing data for the MLOps system.
- *Storage and Versioning* comprises components responsible for storing, versioning, and managing the data and models in the system.
- *ML Training* includes components responsible for training and evaluating the ML models, both in the experimentation and production phases.
- *CI/CD* refers to the category of components responsible for continuously building and deploying ML pipelines, models, and components.
- *Inference* entails the components responsible for providing predictions, making subsequent decisions, and monitoring the system.
- *Infrastructure and Supporting Services* comprises infrastructure components that provide system support, e.g., *Container Orchestrator*, *Workflow Orchestrator*, etc.

Table 3 displays the 39 components by category, their responsibilities, important aliases that we identified for each component name, and the number of papers the component occurred in (from a total of 93 papers).

We combined the synthesized components and their dependencies into a holistic UML component diagram depicted in Fig. 6. In this diagram, we introduce five types of architecture components (see legend). **Baseline components** are core components (white components) in an MLOps system. These components form the foundation of the architecture. They were either present in the majority of studied MLOps architectures or were clearly emphasized to provide key responsibilities in the system, i.e., functionality that is of major importance for MLOps workflows and principles. The more of these baseline components are missing, the less appropriate it becomes to classify the system as an MLOps system. For example, a system with all baseline components except the *ML Metadata Repository* may still be considered an MLOps

system, but removing one or two more baseline components will likely put the system in violation with core MLOps principles.

However, the baseline components need to be complemented with the components of either the **inference service** group (green) or the **inference engine** group (blue) to form a complete MLOps architecture. **Add-on components** represent supplementary components (dotted border) that can be situationally useful or provide complementary capabilities in the MLOps system, e.g., the *Model Comparison Runner*. As a special type of add-on component, the ones of the **online training** group (yellow) can be added to an MLOps architecture, but always together as a group. Note that the *Infrastructure and Supporting Services* components are not included in the diagram, since they provide support to the whole system.

The baseline components include the *Data Preprocessor*, which reads its input from the *Raw Data Store* and stores its results in the *Dataset Repository*. The latter provides data to both the *Data Labeler* and the *ML Experiment Pipeline*. At the same time, the *ML Experiment Pipeline* uses the datasets from the *Dataset Repository* and the ML algorithms from the *Code Repository* to train ML models. It then stores the trained ML models in the *Model Repository* and associated metadata like the training results in the *ML Metadata Repository*. In case of using pre-trained models instead of training models from scratch, the *ML Experiment Pipeline* is substituted with the *Fine-Tuning Module*, which is responsible for further training the selected pre-trained models with domain-specific data to improve their performance in specific tasks.

After deployment, the *Runtime Model Monitor* provides real-time model performance data.

Overall, the blueprint for assembling a complete MLOps system involves the above-mentioned baseline components, as well as incorporating either the inference service or the inference engine group, and potentially the online training group and/or other add-on components. In the following, we describe four characteristic architecture variants (V1 to V4) depicted in Fig. 6.

V1: This variant describes an architecture containing an *Inference Service* to serve ML models in a production environment. The *Inference Service* represents an *ML component* [140], i.e., a containerized, deployment-ready ML model wrapped into an API that is usable for predictions. Thus, in this variant, these ML components are built, tested, and packaged by the *ML Component Builder*, then stored in the *Artifact Repository*, and finally deployed to production via the *ML Component Deployer*.

V2: Contrary to the *Inference Service* of V1, this architecture variant involves an *Inference Engine* to serve the trained ML models. The *Inference Engine* contains a runtime for ML models that allows the dynamic deployment of new models through the *ML Model Deployer*. Hence, this variant efficiently updates only the ML model instead of always replacing the complete ML component. Alternatively, the *Inference Engine* can also check the *Model Repository* to fetch a new model version if certain criteria are met, either periodically or when notified via an event. The inference engine requires a more sophisticated design and implementation compared to the inference service. However, it offers the advantage of enabling a more efficient and optimized inference and enhancing performance.

In variants **V1** and **V2**, the retraining of the models is always manual, i.e., the stakeholders decide to retrain the models or develop new ML models via *ML Experiment Pipeline* based on the data provided by *Runtime Model Monitor*. To use automated retraining of the models, the following variants can be employed.

V3: The third variant combines the components in the *online training group* with V1. The presence of *Trigger* entails the presence of *Continuous Training Pipeline*, *ML Pipeline Builder*, *ML Pipeline Deployer*, and the optional presence of *Model Evaluator*. Essentially, to use the automatic retraining capability provided through the *Continuous Training Pipeline*, we need to provide the packaged code of the ML pipelines and deploy them via *ML Pipeline Builder* and *ML Pipeline Deployer*, respectively. In this variant, the *Trigger*, using data provided by the *Runtime Model*

Table 3

MLOps architecture components, their aliases, responsibilities, number of occurrences, and representative references among primary studies.

	Component name	Aliases	Responsibilities	#	Examples
Data Curation	Data Collector	Data Acquisition, Data Loading	Collects raw data like events from various data sources. As a specialized version, Data Generator generates synthetic data with a certain degree of realism instead of collecting real-world data.	19	S12, S17, S54
	Data Source	External Data Sources	Produces and exposes data from a real-world environment, e.g., domain events, IoT sensors, human inputs, etc.	17	S43, S48, S91
	Data Preprocessor	Data Processing, Data Cleaning, Data Validation, Data Curation Pipeline	Validates, cleans, and prepares collected data for storing as ML training datasets.	25	S11, S36, S57, S79
	Data Quality Checker	Data Quality Assessment, Data Quality Monitoring	Evaluates data quality to ensure it satisfies the defined requirements and criteria.	2	S70, S83
	Data Labeler	Data Annotation, Ground Truth Annotation	Adds the ground truth labels for supervised learning models to dataset instances.	6	S12, S36, S54
Storage and Versioning	Data Catalog	Dataset Catalog	Stores metadata of datasets in an organized inventory, allows users to browse datasets.	4	S15, S57, S70
	Dataset Repository	Data Store, Data Repository, Data Versioning	Stores and versions the datasets used for ML workflows. As specialized versions, Time-Series DB performs as a repository specialized for time-series data and Vector Database Store saves data in a vectorized manner making actions like retrieval more efficient.	38	S17, S41, S57, S75
	Raw Data Store	Data Store	Stores the raw data that are collected from sources. As a specialized version, Data Lake acts as a centralized raw data store that holds all the data.	14	S22, S73, S75, S91
	Feature Store	–	Computes and stores reusable features, serves computed features with low latency.	16	S7, S70, S89
	Code Repository	Source Code Management, Source Code Repository	Stores and versions the training, deployment, and application source code.	19	S55, S72, S92
	Model Repository	Model Registry, Model Store	Stores and versions the trained ML models along with basic metadata, e.g., their versions, etc.	49	S7, S24, S54, S89
	Artifact Repository	Image Repository, Container Registry	Stores a packaged or containerized ML component that incorporates an ML model for inference.	7	S23, S57, S86
	ML Metadata Repository	Experiment Tracking DB, ML Metadata Store	Stores metadata related to model training and model inference for experiment tracking purposes, e.g., model performance metrics, etc. As a specialized version, Model Provenance DB is a comprehensive metadata store that documents the history and creation of ML models.	32	S7, S47, S55, S81
	Feedback Database	Feedback Store	Stores stakeholder feedback and experiences, e.g., from domain experts or engineers, which are manually considered during iterative model development.	3	S12, S28, S54
ML Training	Predictions Storage	–	Records the predictions of the deployed model in inference. This storage can be utilized for tracking predictions and explainability.	2	S47
	Feature Engineering Pipeline	Feature Selection	Selects and transforms the features of the used dataset for model training.	8	S35, S36, S75
	ML Experiment Pipeline	ML Pipeline (Offline), Manual ML Pipeline, Data Science Experiments	Develops, trains, and validates ML models at design time (more experimental and manual).	29	S20, S30, S55, S81
	Continuous Training Pipeline	Continuous Training Pipeline, Incremental Online Learning, MLOps Pipeline	Continuously experiments with and trains ML models at runtime in a production environment (completely automated).	28	R30, S47, S55, S81
	Model Evaluator	–	Evaluates the prediction performance of the models during training.	8	S10, S70, S83
	Fine-Tuning Module	Model Fine-Tuning	Trains the selected pre-trained models with domain-specific data to improve their performance in specific tasks.	2	S63

(continued on next page)

Table 3 (continued).

	Component name	Aliases	Responsibilities	#	Examples
CI/CD	ML Pipeline Builder	Build And Test Pipeline, CI Tool	Builds, tests, and packages (e.g., in containers) the code of the ML pipeline.	6	S24, S79, S86
	ML Pipeline Deployer	Pipeline Deployment, ML Training Pipeline Deployment, CD Tool	Deploys the (built and packaged) code of the ML pipeline to staging or production environments.	5	S24, S30, S79
	ML Model Deployer	Model Deployment, Deployment Pipeline	Deploys the trained model packaged with the dependencies (e.g., required libraries, preprocessing code, etc.) to the production environment.	6	S26, S38, S75, S78
	ML Component Builder	Build Automation Pipeline, Continuous Integration (CI)	Builds and tests deployment-ready ML components by wrapping trained models into containerized artifacts, such as APIs, that can be directly deployed as Inference Services. This component focuses on transforming trained models into reusable, production-ready units, without performing model training itself.	9	S23, S70, S80, S81
	ML Component Deployer	ML Deployment, Continuous Delivery, Continuous Deployment, CI/CD Pipeline	Deploys the ML components (containerized ML models) to staging or production environments.	14	S7, S24, S70, S80
Inference	Inference Service	Model Inference, Production ML Service, Model Server	Serves the trained models to provide new predictions.	26	S36, S49, S63, S76
	Inference Engine	Pool Inference, Local Inference Engine, Model Serving Component	Provides an engine that includes an ML runtime into which trained models can be dynamically deployed to serve predictions.	13	S26, S29, S31, S54
	Runtime Model Monitor	Performance Monitor, Monitoring Component, Model Runtime Monitor	Observes continuously the model-serving performance and infrastructure in real-time, and optionally notifies the responsible stakeholders, or signals the 'Trigger' component based on defined criteria.	32	S44, S54, S55, S75
	Trigger	Retraining Triggering Webhook, Retraining Trigger	Triggers retraining of the ML models based on a predefined performance threshold or predefined events and intervals.	13	S31, S43, S47, S72
	Model Comparison Runner	Model Comparison Runner, Model Metrics Evaluator	Compares the newly trained model to the old model and deploys the better-performing one. This is a post-training component which operates in the production phase.	3	S29, S31, S54
	Decision Processor	Decision Processing	Derives decisions based on the predictions of the model. The decisions are then acted upon by the actors inside or outside the system.	3	S23, S36, S75
	XAI Module	Explainability Module	Provides interpretable explanations of model decisions to support transparency, trust, and regulatory compliance across the MLOps lifecycle.	2	S68, S81
Infrastructure and Supporting Services	Resource Manager	Resource Leasing, Model Engine	Provides foundational hardware and software computational resources. The provided computational resources can be distributed or non-distributed and scalable or non-scalable.	6	S2, S7, S36, S86
	Communication Middleware	Message Queue, Event Streaming Bus	Distributes the received requests and model predictions to resources. As a specialized version, MQTT Broker acts as a lightweight, publish-subscribe messaging protocol that handles requests as messages.	6	S23, S31, S48, S59
	Container Orchestrator	Container Service, Container Manager	Bundles the model with all of its related files, configurations, and dependencies to run it efficiently across different environments running the same container's engine.	3	S6, S24, S69
	Workflow Orchestrator	Adaptive Scheduler, Workflow Orchestration, Job Tracking Module	Provides system-wide orchestration, decides the execution schedule of multiple models balancing throughput and latency and informs the data requirement of each model to the data coordinator.	12	S7, S11, S48, S92
	Log Manager	Logging, Info Collector, Object Store, Predictions Store	Records and saves the information about running the services, training, user requests, predictions, etc.	6	S29, S31, S44
	API Gateway	–	Provides interaction between the components within the platform and also between the platform and external entities.	8	S15, S29, S86
	MLOps User Interaction Manager	Ops Dashboard, Front-End	Provides interaction between the users and the MLOps platform. This component includes the visualization tasks such as data characteristics and monitoring results (data, model, ...).	10	S44, S54, S55

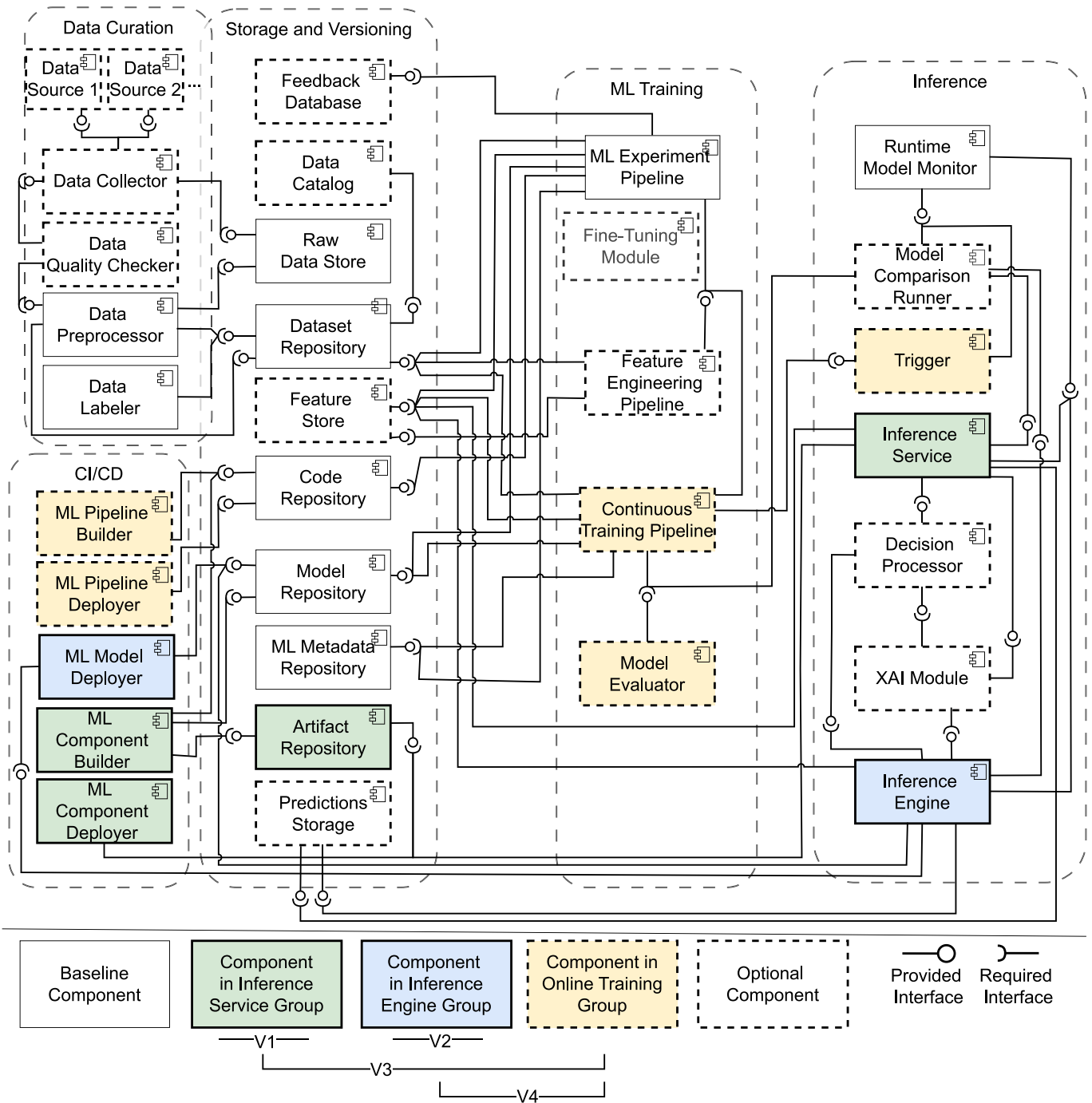


Fig. 6. UML component diagram of MLOps architecture variants.

It should be noted that in case of using pre-trained models, the *ML Experiment Pipeline* component will be replaced by *Fine-Tuning Module*, preserving the same connections and interfaces which leads to variants V5-V8.

Monitor, submits a periodic or event-based retraining request to the *Continuous Training Pipeline*. This automatically retrains a new model and then builds and deploys a new *Inference Service* in production.

V4: The fourth variant, similar to the third, combines the *components in the online training group* with V2, which results in the automatic retraining and deployment of ML models in the *Inference Engine*.

V5-V8: These variants correspond to V1-V4, respectively, with the distinction that they utilize pre-trained models rather than training the models from scratch.

Vx: In addition to these eight described variants, the selection of any combination of the add-on components can result in several additional variants. Table 3 can be consulted for a detailed description

of all the other components. As an example, we describe the addition of the *Model Comparison Runner*: adding this component to either V3 or V4 allows more informed model update decisions. The *Model Comparison Runner* compares the performance of a newly trained model in the production environment to the currently deployed models and only deploys it if it performs better than the deployed models. In the absence of this component, the newly trained model is always deployed, even if its performance would be inferior to the current ones.

If we consider the 11 add-on components and the eight major variants described, the selection of any combination between them results in a large number of different architecture variants. Selecting the most suitable variant may depend on factors like specific design decisions, resource availability, scalability considerations, required update frequency, or technological expertise. Even though we provide the conceptualization of MLOps architectures in this paper, selecting the most suitable variant would also rely on guidelines for practitioners which, as described in the introduction, we are planning to provide as a future work.

Key insights from RQ1.1

1. The description of MLOps systems in the literature comprises domain-specific as well as domain-agnostic architecture components, spanning 6 major categories, from data curation to inference, plus infrastructure & supporting services.
2. The selection of MLOps architecture components is highly dependent on the requirements, available resources, and, eventually, the architectural design decisions of the architecture team.
3. Using the provided architecture variants (Fig. 6), companies can select the best architecture variants based on their requirements and resources.

4.2. Tools used to support or implement the components (RQ1.2)

From the 93 primary studies, we identified a total of 136 tools and platforms. Fig. 7 shows a heatmap of the tools and platforms that are mentioned at least 5 times in the papers, mapped to the components they support. The figure shows that *MLflow* (the far-right column) is the most frequently mentioned platform (21 mentions). This tool is mentioned to support the end-to-end ML lifecycle, and as also evident in the figure, it is mentioned in the papers to support a wide variety of components in an MLOps system. Jenkins follows with 16 mentions for supporting the components within the *CI/CD* category. *AWS SageMaker* is also one of the most popular tools (15 mentions). This tool is observed to support the highest number of components in the architecture of MLOps workflows (10 distinct components among 5 categories of our architecture). *Kubernetes* follows by supporting 3 different components with 11 mentions.

Within the six categories of architecture components, the greatest variety of tools is mentioned for *Storage and Versioning*, followed by *CI/CD*. Some of the tools supporting the entire *CI/CD* category encompass *Jenkins*, *GitLab*, *Bamboo*, *GitHub Actions*, *BitBucket*, *TeamCity*, and *CircleCI*. Regarding the components, however, the most number of tools are mentioned for *Runtime Model Monitor*, *Model Repository*, and *Workflow Orchestrator*. There are also several components for which no tool is mentioned. These include *Data Catalog*, *Predictions Storage*, *Feedback Database*, *Data Quality Checker*, and *Inference Engine*. The complete mapping between the tools and the components can be accessed through the replication package [141].

Lastly, some papers generally mention tools and services that support the end-to-end MLOps workflows, without mapping them to any

specific component. These tools include *AWS SageMaker*, *MLflow*, *KubeFlow*, *Weights & Biases*, *Clear ML*, *MLReef*, *Iguazio*, *Polyaxon*, *Vertex AI*, and *Azure ML*.

Key insights from RQ1.2

1. The development and assembly of a complete MLOps system requires the integration of multiple tools by different vendors.
2. Although several end-to-end tools claim to be applicable across the entire MLOps system, our study reveals that each tool is typically used to implement specific components rather than the entire system.
3. While some components are supported by many existing tools, for other components, no specific tools are explicitly mentioned, indicating opportunities for tool vendors.

4.3. MLOps process steps and their sequence (RQ2.1)

From the 93 primary studies, we identified and synthesized 56 unique process steps comprising **actions** (atomic steps in the completion of a process [142]) and **activities** (composite steps including several actions) that are performed within MLOps. These steps comprise the end-to-end process from identifying and analyzing a business problem to monitoring and maintaining the ML models and the whole MLOps system. We group the identified steps into 7 categories:

- *Requirements Engineering* entails steps that are performed to extract and analyze the business and technical requirements and to model the identified business problem in a way that makes it addressable by an ML solution.
- *Architecture Design and Implementation* refers to the category of steps that design and implement the holistic MLOps system. These steps address the overall system and concern software aspects.
- *Data Engineering* comprises the steps for collecting, annotating, manipulating, managing, versioning, and storing the data in the MLOps system.
- *ML Model Design And Implementation* refers to the steps that design and implement the code of the ML models in the offline environment. These steps specifically address the models themselves and are not concerned with the overall system architecture or design.
- *Model Training* comprises the actions that train ML models in the offline and online environments, from feature calculation to registering the trained models in the model repository.
- *CI/CD* entails actions that ensure proper integration of the ML models and the software (non-ML) components via continuous integration and continuous deployment.
- *Inference* comprises actions performed for and around model predictions and performance in the production environment, including model monitoring.

Table 4 displays the actions and activities per category, the important identified aliases for each, their descriptions, responsible roles, and number of occurrences of each in the reviewed papers.

We synthesized the 56 identified steps into the UML activity diagram depicted in Fig. 8. This activity diagram visualizes the process view of our synthesized architecture of MLOps systems (an end-to-end process of designing, implementing, operating, and maintaining such systems). Similar to the components of the structural view, the process view contains core and add-on actions. Core actions are advised to be performed in the process of any end-to-end MLOps system, e.g., *Design Software Architecture* and *Design ML Model*. Add-on actions, on the

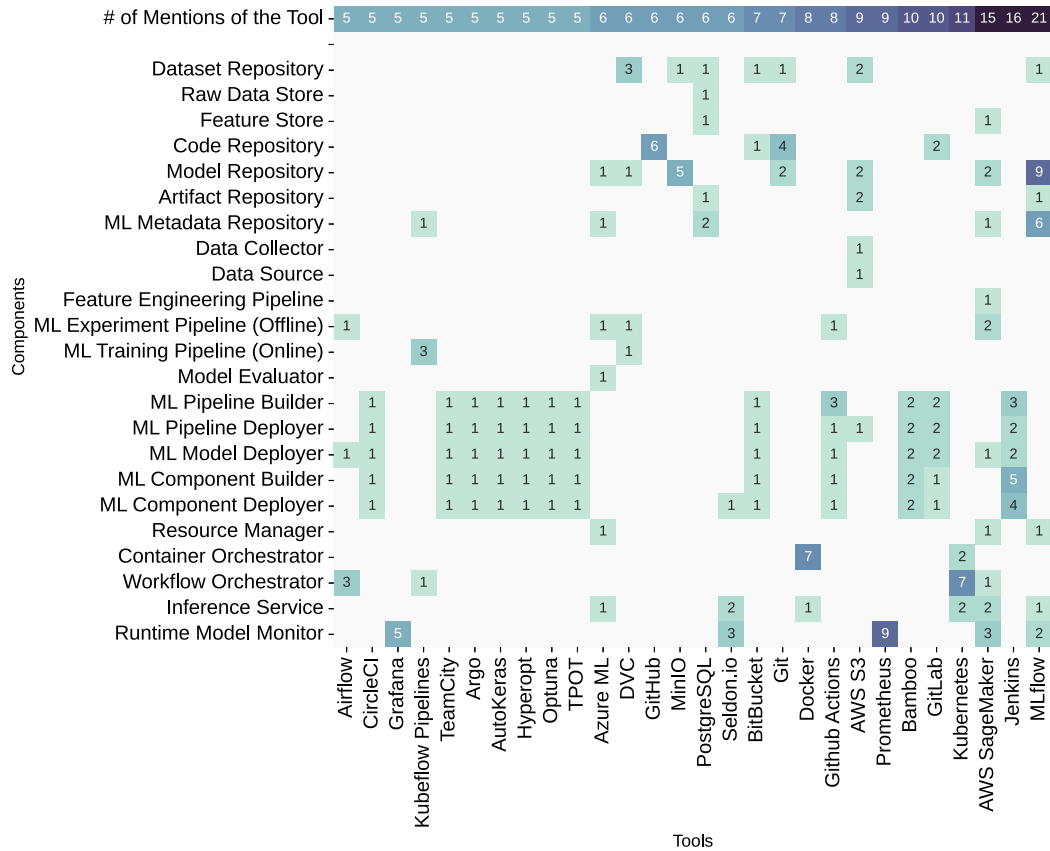


Fig. 7. The most popular tools and platforms, mapped to components, ordered by # of mentions.

other hand, may only be performed at the discretion of the MLOps architecture team, e.g., *Collect Data* and *Collect Learner Feedback*.

In the following, we first describe the complete high-level process and then we zoom into the details of its more complex activities.

The complete process of an MLOps system starts with the *Requirements Engineering* category of actions. Beginning with *Analyze Business Problem*, the domain expert identifies or analyzes the business problem and requirements that can be solved by ML, and describes the use case to define the business goal. Once the business goal is defined, the data scientist *Explores ML-based Solutions*, i.e., he or she models the analyzed business problem from an ML perspective, e.g., by specifying which ML tasks (regression, classification, clustering, etc.) can be used to address the identified business problem. Next, the data scientist and ML engineer *Define Solution Requirements*, which are the system-level requirements, e.g., ML model structures, data requirements, security aspects, infrastructure, etc. As the last action in this category, the business analyst *Analyzes Solution Requirements*. By analyzing the defined requirements, the business analyst assesses the feasibility of addressing them and breaks them into detailed and actionable specifications to be fulfilled by the MLOps system. The solution requirements may be ML-specific requirements, data requirements, or software functional and non-functional requirements.

After *Requirements Engineering*, the following sets of actions are performed simultaneously:

In *Application Design and Implementation* category, the solution architect specifies and *Designs System Architecture*. This action entails the architectural design decisions for the whole system (e.g., the components and their interaction). Once the system architecture is designed, the software engineer *Implements Non-ML Parts* of the MLOps system, i.e., its non-ML components.

In parallel to the design and implementation of the non-ML components, data is collected and ML experimentation is performed. First, the domain expert and data engineer optionally *Collect Data* from sources, e.g., sensors or APIs. This are raw data that need to be preprocessed and validated. In this step, the domain expert and data engineer may also *Generate Data*, i.e., produce new synthetic data via e.g., simulation, or *Augment Data*, i.e., produce more data by duplicating and modifying existing data, e.g., by zooming the images of an image dataset. Then, the data scientist and the data engineer *Analyze the Data* to identify characteristics such as data distribution, patterns, and correlations. In this step, if the data scientist and data engineer need more or different data, the two actions *Collect Data* and *Analyze Data* are repeated until the collected data is satisfactory. These two actions are omitted in the MLOps systems where stakeholders already have access to collected data through a *Dataset Repository*. The data scientist and data engineer then *Select Data* to be prepared for further use during model training — here they may choose to only continue with a subset of the data available to them. Then, they can *Preprocess the Data* by applying data transformations, cleaning and normalization. We have seen this action being also optionally done during the training process later on. The domain expert then *Labels Data* to annotate the ground truth data to be used in the training of the ML models, and the data engineer *Versions* and *Stores* this *Data* in the dataset repository. Once the data is stored, the *Perform ML Experimentation* activity begins, where different stakeholder roles perform different actions to design, implement, and train ML models (see Section 4.3.1).

Upon completion of the design and development of the non-ML components and the trained ML models, the following actions are performed in parallel to transition the MLOps system from the development environment into the production environment:

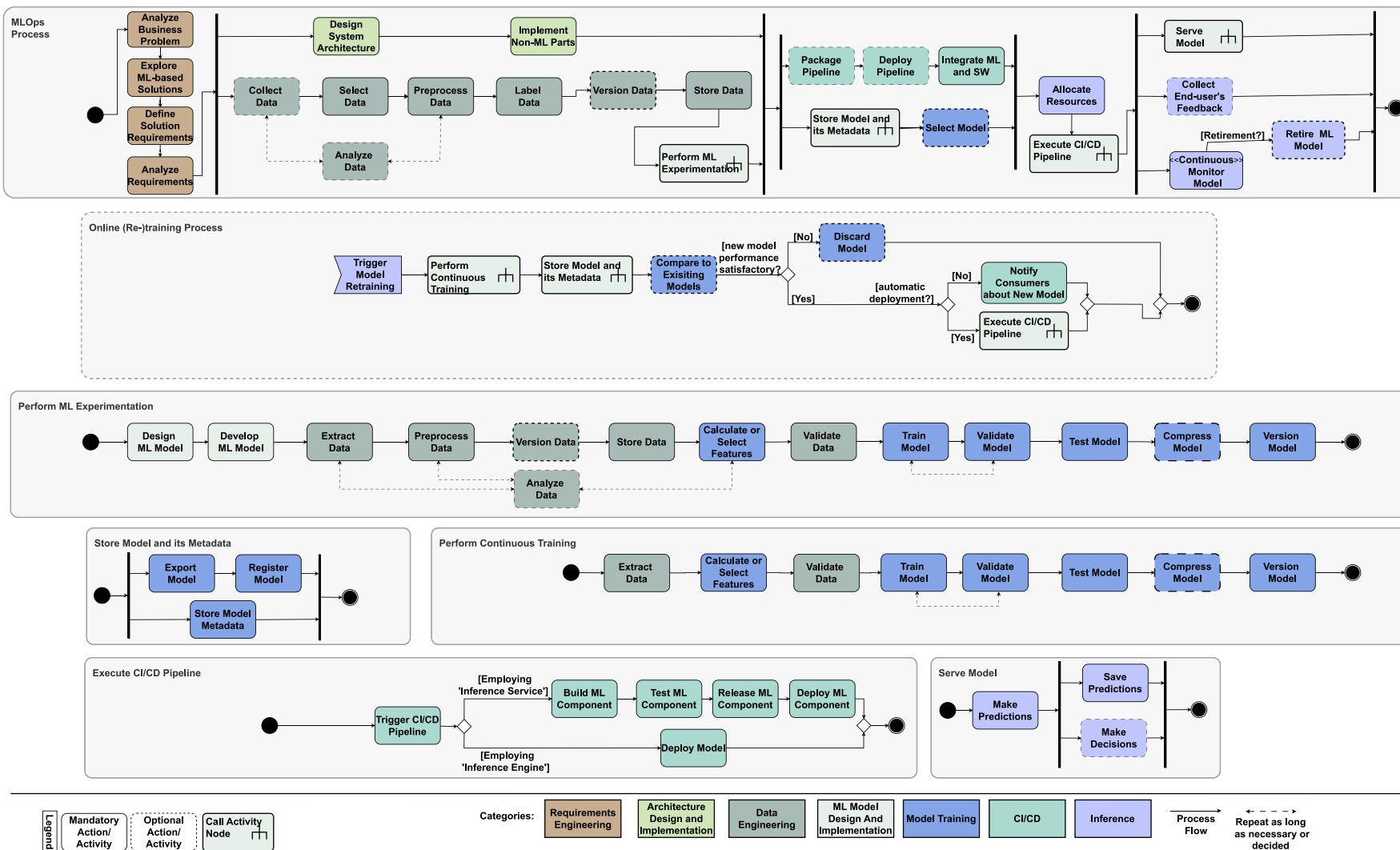


Fig. 8. UML activity diagram of MLOps architecture.

Table 4

MLOps process actions and activities (the latter are highlighted with *), their aliases, description, responsible roles, number of occurrences, and representative references among primary studies.

	Action/ Activity*	Aliases	Description	Stakeholders	#	Examples
Requirements Engineering	Analyze Business Problem	Business Problem Analysis, Domain Analysis, Understanding and Framing the Problem	Identifies and analyzes a business problem that can be solved by ML and describes the use case. The output is the defined business goal. As a specialized version, stakeholders can Search for Similar Problem in a Problem DB, and generate model strategy based on similarities.	Domain Expert	11	S7, S16, S52, S71
	Explore ML-based Solutions	Model ML Problem, Derive ML Problem from Business Goal	Models the analyzed business problem in ML, e.g., specifies which ML tasks(regression, classification, or clustering) can be used to address the defined business goal.	Data Scientist	11	S7, S71, S74, S84
	Define Solution Requirements	Business Requirements Definition, Project Requirement Gathering and Definition	Defines the MLOps system requirements, e.g., ML model structures, data requirements (e.g., what type of data and the sources of data), security aspects, infrastructure, etc.	ML Engineer, Data Scientist	11	S21, S35, S71, S74
	Analyze Solution Requirements	Requirements Analysis	Analyzes the defined requirements, assesses the feasibility of addressing them, and breaks them into detailed and actionable specifications to be fulfilled by the MLOps system. This step includes analyzing solution requirements, data requirements, and software functional and non-functional requirements. Failure and Deficiency Analysis, and Hazard and Risk Analysis are specialized versions of this action focusing only on the named specific matters.	Business Analyst	8	S35, S71, S78, S84
Architecture Design and Implementation	Design System Architecture	Architectural Design, System Specification, Functional and Technical Architecture Description	Specifies and designs the architecture of the MLOps system. This action entails the architecture design decisions and implementation decisions. Safety Concept Design is a specialized version addressed in critical domains.	Solution Architect	14	S7, S16, S71, S78
	Implement Non-ML Parts	Non-ML Software Implementation, Code Non-ML Parts	Implements and performs unit tests on the non-ML components of the MLOps system.	Software Engineer	8	S16, S78, S88
Data Engineering	Collect Data	Stream Data Ingestion, ML Data Acquisition, Ingestion	Collects raw data from sources, e.g., sensors, for training the ML model, and saves the data in 'Raw Data Store component'. This action may involve designing and implementing the necessary data pipelines for data ingestion. As specialized cases this action can Get Similar Dataset or Build Dataset where data from a datalake is used, Generate and Augment Data where synthetic data is produced by for example altering the existing data in different forms, and Extraction of Additional Information from the inference when models are running.	Domain Expert, Data Engineer	40	S12, S17, S80, S81
	Select Data	Apply Query	Queries and selects desired data to be used in further actions, e.g., to be preprocessed, labeled, etc.	Data Engineer	1	S80
	Extract Data	Data Extraction, Data Ingestion, Data Fetching	Fetches required data for training the ML model from available databases in the system.	Data Engineer, Data Scientist	22	S7, S23, S35, S73
	Preprocess Data	Data Preprocessing, Data Transformation And Cleaning, Prepare Data	Applies preprocessing steps on data like data cleaning, data transformation, data normalization and standardization, etc. to prepare data for training ML models. As a specialized version, Transformation and Cleaning Rules may also be defined in this step before preprocessing the data.	Data Scientist, Data Engineer	66	S23, S56, S69, S72
	Analyze Data	Exploratory Data Analysis, Pattern Analysis, Data Search/ Analysis	Analyzes the data to identify data characteristics such as data distribution, patterns, correlations, etc. Analysis results affect stakeholders' decisions in the 'Prepare Data' and 'Calculate or Select Features' steps. Data characteristics are visualized via the 'MLOps User Interaction Manager' to facilitate this step.	Data Scientist , Data Engineer	24	S7, S17, S53, S81
	Label Data	Data Annotation, Labeling Pretreatment, Training Data Creation	Provides labels for ground truth data to be used in training the models.	Domain Expert	7	S12, S17, S71
	Version Data	Data Versioning	Versions the collected and labeled or the preprocessed data for tracking and provenance purposes.	Data Engineer	5	S12, S21, S53, S80
	Store Data	Data Storage	Stores the collected and labeled or the preprocessed data in Dataset Repository.	Data Engineer	9	S12, S53, S72
	Validate Data	Data Validation, Data Sanity Test, Data (Product) Quality	Assesses and assures the quality of extracted data, e.g., data integrity, distribution, and feature relevance, before using the data for training ML models.	Domain Expert, Data Scientist, Data Engineer	12	S7, S49, S71, S73
ML Model Design and Implementation	Design ML Model	Training Algorithm Management, Algorithm and Hyperparameter Selection, Generate Model Development Strategy, Problem/Pipeline Structure Search	Designs the ML pipeline structure. The design depends on the type of data, the specified ML problem, and specific performance metrics demands.	ML Engineer , Data Scientist	20	S16, S21, S37, S71
	Develop ML Model	Development, Code ML, Model Programming	Implements the code of the designed ML model to be trained and saves it in the 'Code Repository'.	Data Scientist, Software Engineer	19	S44, S71, S74, S78
	Perform ML Experimentation*	Experimentation	Extracts and prepares data, and trains the developed ML model with training data. This action is performed manually in the offline (experimentation) environment and provides a trained ML model as the result.	Data Scientist	19	S7, S53, S74, S84

(continued on next page)

Table 4 (continued).

	Action/ Activity*	Aliases	Description	Stakeholders	#	Examples
Model Training	Calculate or Select Features	Feature Engineering Pipeline, Generate or Use Cached Feature Pipelines, Feature Selection	Calculates new, more complex features based on currently available features and data and/or selects currently available features for model training. As a specialized version, Feature Engineering Rules may also be defined in this step to which feature calculations should adhere to.	Data Scientist , Data Engineer	14	S11, S74, S91
	Train Model	Model Training, ML Pipeline Execution, Model Building, (Re-)Train, Model Development	Trains an ML model by adjusting the extracted data, tuning the hyperparameters, and model parameters. This action is iterative.	Data Scientist	56	S7, S16, S44, S37
	Perform Continuous Training*	Online Pipeline, Continuous Training, Automated ML Pipeline	Comprises actions from data extraction from the 'Dataset Repository' to versioning the trained model ready to be exported and registered in the 'Model Repository'.	System	11	S7, S9, S30, S74
	Validate Model	Model Performance Management, Model Validation, ML Model Verification, Model Analysis, Model Testing	Evaluates the trained model performance on the validation data and returns to 'Train Model' to adjust the hyperparameters in training until expected performance is reached or thresholds are met.	Data Scientist	42	S7, S21, S72
	Test Model	Model Testing, Review, Model Assessment, Verify, Model Evaluation, Model Validation and Governance	Tests the model with respect to different metrics such as model performance, effectiveness, prediction duration, resource consumption, etc., by making predictions on unseen data before releasing the model for deployment. Model performance metrics may be optionally visualized via the 'MLOps User Interaction Manager' component.	Data Scientist, Domain Expert	33	S23, S26, S72, S87
	Version Model	Model Versioning, Versioning, Model Version	Versions the trained ML model with hyperparameters, data version dependencies, performance demands at training time, time of training the model, etc.	Data Scientist	6	S21, S49, S88
	Store Model Metadata	Register Metadata, Experiment Tracking	Saves the metadata related to training an ML model, e.g., hyperparameters, performance metrics, etc.	Data Scientist	9	S23, S31, S45, S74
	Export Model	Export Model, Model Packaging	Exports the trained ML model in a serializable format to be saved in the 'Model Repository'.	Data Scientist	2	S7, S20
	Register Model	Push Model To Repository, Model Storage, Model Releasing, Publish Model, Integrating Assets with Repository	Saves the exported ML model in 'Model Repository' component.	Data Scientist	19	S7, S20, S29, S53
	Store Model and its Metadata*	–	Exports the model and registers the model in 'Model Repository'. In parallel, optionally stores the model metadata in 'ML Metadata Repository'.	Data Scientist	5	S88
	Select Model	–	Selects the most suitable trained model to be deployed into production.	–	5	S37, S53
	Compare to Existing Models	–	Compares the new model with existing ones using different strategies, and either deploys or discards the new model.	Data Scientist, DevOps Engineer	3	S74
	Discard model	Keep Old Model	Discards the newly trained model if its performance is not satisfactory compared to the already deployed models.	–	2	S68
	Compress Model	–	Builds a compact and optimized model which is suitable for scaling. Quantization is a specialized version of this action where the precision of model parameters is reduced in favor of decreasing the model size and increasing inference speed.	Data Scientist, Software Engineer	7	S26, S27, S74
Model Training	Select Suitable Pre-Trained Model	Model Selection, Pre-Trained Model Selection	Analyzes and selects the best pre-trained model or base model that best suits the analyzed requirements and defined goals. As a specialized version, a Foundation Model may be selected which is a very larger and often general pre-trained model.	–	5	S71, S82
	Perform Fine-tuning	Model Finetuning, Foundation Model Customization and Fine-tuning	Fine-tunes the selected pre-trained or foundation model on domain-specific data to improve task performance regarding the specific requirements and goals.	–	4	S67, S82
	Perform Prompt Engineering	Prompt Engineering	Designs and optimizes prompts to guide model behavior for specific requirements and tasks.	–	4	S67, S82
	Perform In-Context Learning	Model Training	Provides a model with input-output examples for a specific task (like translation, classification, or arithmetic) so that the model can use those examples as context to infer the task pattern and generate the correct output for new inputs.	–	2	S82

(continued on next page)

Table 4 (continued).

	Action/ Activity*	Aliases	Description	Stakeholders	#	Examples
CI/CD	Integrate ML and SW	Model Integration into Application, ML/SW Integration	Integrates the trained and validated ML model and the non-ML components and performs unit tests on the integration to create the overall MLOps system.	Software Engineer	9	S16, S21, S67, S71
	Trigger CI/CD Pipeline	CI/CD Pipeline Triggering	Triggers the CI/CD pipeline to continuously retrain, integrate, deliver, and deploy ML models while maintaining traceability. The trigger can be data-driven; upon arrival of new data, schedule-driven, or metrics-driven; e.g., upon degradation of model performance.	System	4	S20, S64, S74
	Package Pipeline	Build, Test, and Package ML Pipeline, Pipeline Continuous Integration	Packages ML pipelines (developed ML training code) with its dependencies, e.g., libraries, configurations, etc. to be used in (re-)training ML models in production environment.	DevOps Engineer, ML Engineer	4	S30, S35
	Deploy Pipeline	Upload And Run, Deploy ML Pipeline, CD: Pipeline Deployment	Deploys the packaged ML pipeline in the production environment.	DevOps Engineer	5	S30, S35
	Deploy Model	Solution Deployment, Model Deployment and Insight Consumption, Deploy for Serving	Deploys the trained and validated ML model into production environment to be served.	DevOps Engineer, ML Engineer	23	S37, S49, S78
	Build ML Component	Build, Build Image, Build Artifacts, Model Packaging, Package	Builds ML components or software artifacts of the trained ML model.	DevOps Engineer, ML Engineer	21	S26, S37, S52, S65
	Test ML Component	Test Artifact, Unit and Integration Tests, CI/CD Validation, Verify	Tests the ML component or software artifact before saving the component in 'Artifact Repository'.	DevOps Engineer, ML Engineer	16	S21, S37
	Release ML Component	Deliver Artifact, Release, Push, Release Artifacts	Saves the ML component in 'Artifact Repository'.	DevOps Engineer, ML Engineer	5	S20, S23
	Deploy ML Component	Deploy to Production, HW/SW Integration, Continuous Deployment	Deploys the packaged ML component or software artifact into production.	DevOps Engineer, ML Engineer	22	S52, S73, S87
	Notify Consumers about New Model	Notify Streaming Consumers about New Model Version, Streaming Consumers Request New Model Version	Notifies the consumers that a new trained ML model is available upon completion of the (re-)training action.	System	3	S29, S73
Inference	Execute CI/CD Pipeline*	–	Triggers the CI/CD pipeline and prepares and deploys the 'ML training pipeline', and either 'ML Model' or 'ML Component' depending on the selected architecture variant.	DevOps Engineer	17	S50, S67, S74
	Allocate Resources	Resource Allocation, Resource Leasing	Allocates resources for executing ML training experiments or running trained ML models in production. As a specialized version, Estimation of Resources estimates the resource consumption based on different criteria and e.g., time of the day.	Business Analyst	6	S27, S56, S78
	Serve Model*	Model Serving, Model Inference, System Operation in the Field, Make Predictions Based on Streaming Events	Serves the trained and validated ML models in production to provide ML-based solutions, make and save predictions and optionally make decisions. As specialized versions, Field Test and Stage Deployment refer to running and evaluating the model in a staging environment before deploying to production. This step may also include collecting the feedback of the users.	System	31	S16, S29, S72, S85
	Make Predictions	–	Provides the end-users with ML predictions on provided data at inference.	System	4	S29, S85, S91
	Trigger Model Retraining	Retraining Trigger, Align, Monitoring and Retraining Model, Triggering	Triggers ML training pipeline to retrain models in production environment based on model degradation (result of monitoring metrics), user request, time intervals, or arrival of new data.	System, DevOps Engineer	15	S31, S49, S72, S74
	Save Predictions	–	Saves model predictions for future use, e.g., for calculating model accuracy based on predictions and provided ground truth labels for the same data. As a specialized version Match Predictions to Ground Truth is performed to collect new data from the same system for the new trainings.	System	3	S29
	Make Decisions	Decision Processing	Processes the predictions and makes decisions based on the ML model predictions, e.g., terminates a process, or closes a valve in manufacturing based on the predictions.	System, End-user	2	S36, S91
	Monitor Model	(Model) Monitoring, Runtime Monitoring and Data Collection, Production Monitoring	Monitors different metrics of the deployed model, and triggers retraining, decommissions the deployed model, or notifies the responsible stakeholders about performance degradation.	System, DevOps Engineer, ML Engineer, Domain Expert	53	S16, S37, S66, S67
	Collect End-user's Feedback	Learner Progress, Learner Feedback	Collects the feedback of the end-users of the MLOps system and provides it as new collected data.	Domain Expert	2	S12
	Retire ML Model	Model Retirement	Withdraws/ Retires the ML Model when ongoing monitoring detects certain conditions, e.g., when violations of the AI Act compliance criteria are detected.	–	1	S84

- The DevOps engineer and ML engineer optionally *Package ML pipelines* (developed ML training code) with their dependencies, e.g., libraries, configurations, etc., to be used in (re-)training ML models in the production environment. Subsequently, the DevOps engineer *Deploys* the packaged ML Pipelines in the production environment. The software engineer then *Integrates ML and SW*; they integrate the trained and validated ML models and the non-ML (SW) components to create the overall MLOps system.
- Simultaneously, the activity *Store Model and its Metadata* is performed, where the data scientist *Exports* the trained ML Models in a serializable format, and then *Registers* the Models by saving the exported ML models in the *Model Repository* component. In parallel to these two actions, *Store Model Metadata* saves the metadata related to training each of the ML models, e.g., hyperparameters, performance metrics, etc. It should be noted that, throughout the system, multiple ML models may be trained simultaneously, with their pipelines and data stored accordingly. The data Scientist then compares the different trained models based on specific required criteria and *Selects* the most suitable trained Model(s) to be deployed into production.

The ML owner, as a specialization of the business analyst role, then *Allocates Resources* for running trained ML models in production. In the absence of an ML owner role, the business analyst performs this task in the MLOps system.

The system then *Executes CI/CD Pipeline* to deploy the trained models in production. This activity comprises three different sets of actions, which are described in Section 4.3.2.

Lastly, the following steps of the *Inference* category are performed in parallel:

- Through the activity *Serve Model*, the trained and validated ML models are served by the MLOps system in production. This is realized through three actions: first, the system *Makes Predictions* to provide the end-users with ML predictions on unseen, user-provided data. Afterward, it *Saves the predictions* for future use, e.g., for calculating model accuracy based on predictions and post-hoc provided ground truth labels for the same data. Simultaneous to saving the predictions, the system may also *Make Decisions* by processing the predictions and making decisions based on them, e.g., terminates a process or closes a manufacturing valve. If this action is not automated, the end-user makes the decisions based on the provided predictions.
- In parallel to model serving, the domain expert optionally *Collects the End-User's Feedback* for the MLOps system and provides it as newly collected data.
- During *Inference*, the system automatically and continuously *Monitors* the Models, and *Triggers Model Retraining* based on defined thresholds. The DevOps engineer and ML engineer can also *Monitor* the Models using the metrics provided by the system. Moreover, the domain expert provides insights to the data scientist regarding the performance of the models during this continuous action. During the monitoring action, it may also be decided for a specific model to be *Withdrawn* or *Retired*, i.e., when ongoing monitoring detects certain non-acceptable conditions, such as concrete violations of the EU AI Act.²

The monitoring can result in the *Online (Re-)training* activity being executed. This activity is invoked by the receiving signal *Trigger Model Retraining*, where the system automatically triggers the ML training pipeline to retrain models in the production environment. Triggers can be based on model degradation identified through monitoring metrics, but also based on user requests, time intervals, or the arrival of new data. The DevOps engineer may also trigger the retraining manually if necessary. Once the retraining pipeline is triggered, the activity *Perform Continuous Training* is executed, which is described in Section 4.3.3.

After the models are automatically trained in this stage, *Store Model and its Metadata* is performed to export, register, and save the models and their related metadata. The newly trained models are then

Compared to the Existing Models currently deployed to production. If a newly trained model underperforms relative to an existing one, it is *Discarded*. Otherwise, the system proceeds in one of the following ways: it either *Notifies the Consumers* that a newly trained ML model is available (in case there is no continuous deployment), or it *Executes the CI/CD Pipeline* to automatically deploy the model.

4.3.1. Perform ML experimentation

This activity involves taking steps from designing an ML model to versioning the trained model, ensuring it is properly prepared for storage in the *Model Repository*. First, the ML engineer and data scientist *Design* the ML Model or the ML pipeline structure, which depends on the type of data, the specified ML problem, and specific performance metrics demands. Then, the data scientist and the software engineer *Develop* the ML Model: they implement the code of the ML model to be trained (usually with the help of ML libraries), and save the training code in the *Code Repository*. Once the ML code is ready, the data engineer and the data scientist *Extract Data*: they fetch the required data for training the ML model from the previously prepared datasets. Then, they *Preprocess* the Data, i.e., they apply preprocessing steps such as data cleaning, data transformation, data normalization, and standardization, to prepare the data for training ML models. The data engineer then *Versions and Stores* the preprocessed data in the *Dataset Repository*. As described earlier, *Preprocess Data*, and consequently *Version and Store Data* may also be performed at an earlier stage, directly after *Collecting* the raw Data. Using and analyzing the preprocessed data, the data scientist and the data engineer *Calculate* new, more complex *Features* based on currently available data and/or *Select* currently available *Features* for model training. It should be noted that the actions of *Extract Data*, *Preprocess Data*, and *Calculate or Select Features* are applied iteratively: the data scientist and the data engineer continuously analyze the data and, if needed, extract more or different types of data, apply different preprocessing techniques, or redefine the feature selection. The selected features are then passed through *Validate Data*, where the domain expert, the data scientist, and the data engineer assess and ensure the quality of extracted data, e.g., data integrity, distribution, and feature relevance, before using the data for training ML models.

Once the data are preprocessed and validated, the data scientist iteratively *Trains the Models* by adjusting the extracted data and tuning the hyperparameters and model parameters, and *Validates the Models*, i.e., their prediction performance. The iteration is repeated until the expected performance is reached. Then, the data scientist and the domain expert *Test the Models*: they evaluate the performance of the models on unseen data ("holdout data"), but also evaluate other aspects such as inference speed and resource consumption before releasing them for deployment. At this stage, the data scientist and the software engineer may *Compress the Models*, i.e., they build a compact and optimized version of each model, which is suitable for scaling, e.g., the precision of model parameters may be reduced in favor of decreasing the model size and increasing inference speed. Before being deployed, each trained model is *Versioned* with hyperparameters, data version dependencies, performance demands at training time, time of training the model, etc.

4.3.2. Execute CI/CD pipeline

This activity begins with an automatic action that *Triggers the CI/CD pipeline* to continuously retrain, integrate, deliver, and deploy ML models while maintaining traceability. The trigger can be data-driven, i.e., when new data becomes available in the system; schedule-driven; or metrics-driven, for instance, when the model performance declines.

During this activity, either an ML component or an ML model is deployed. The former case is when the MLOps system contains an *Inference Service* to provide model predictions (see Section 4.1). In this case, four actions need to be performed: the DevOps engineer and the ML engineer *Build the ML component* or software artifact of the trained ML model, and *Test the ML component* or software artifact. Afterward,

² <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>

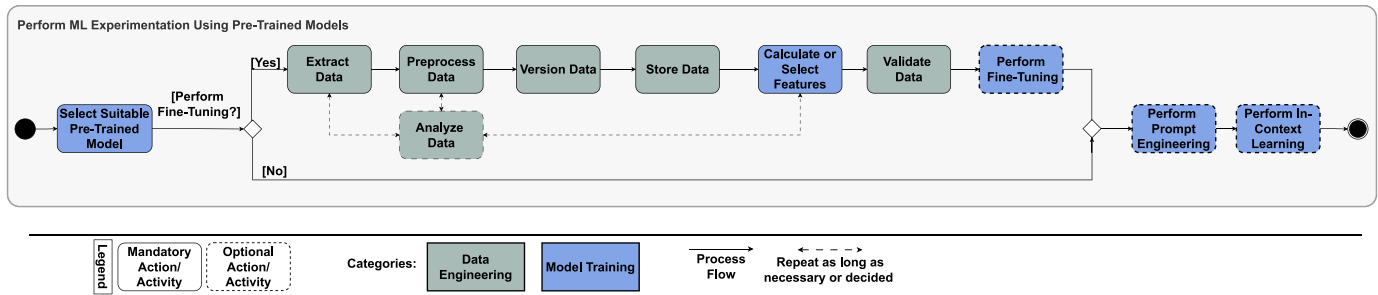


Fig. 9. Using a pre-trained model in ML experimentation.

they *Release the ML Component* and save it in the *Artifact Repository*, and lastly, they *Deploy* the packaged *ML Component* or software artifact to production.

The latter case is when an *Inference Engine* is used in the MLOps system. In this case, the DevOps engineer and the ML engineer only *Deploy* the trained and validated *ML Model* for serving it to the production environment. It should be noted that the ML model or the ML component may first be deployed into the staging environment and then deployed to production, in case the performance is satisfactory (e.g., in R34).

4.3.3. Perform continuous training

This activity is entirely automated and closely resembles the *Perform ML Experimentation*. However, *Online ML Training* reuses the (code of the) ML models that is developed in *Offline ML Experimentation* rather than developing new models. During this automatic activity, the system *Extracts Data*, *Calculates or Selects Features*, and *Validates Data*. When the data is prepared, an iteration of *Train Model* and *Validate Model* is repeated until evaluation metrics are satisfied. Lastly, the system *Tests the Model*, optionally *Compresses the Model*, and *Versions the Model* ready to be stored in the *Model Repository*. It is important to stress that the system itself (instead of the user) may perform a number of automated trials (experiments) that involve training the existing models with different hyperparameters, provided that such experiments are fully automated.

4.3.4. Perform ML experimentation using pre-trained models

Generally, we visualize our structural and process views of the MLOps system assuming that the MLOps team designs and implements the whole system. However, with the rise of pre-trained models such as foundation models, one possibility is to use such models as a base and adapt them to the context and requirements of the system at hand. In this case, a suitable pre-trained model is selected instead of designing and developing an ML model from scratch. In other words, instead of the *Perform ML Experimentation* activity, we will have a new activity *Perform ML Experimentation Using Pre-Trained Models*.

This activity, as visualized in Fig. 9, begins by *Selecting a Suitable Pre-Trained Model* step in which the best pre-trained model or base model that best suits the analyzed requirements and defined goals is analyzed and selected. The pre-trained model can be any model from a smaller context-dependent model to a foundation model, which is a very large and general pre-trained model. The selected pre-trained model may be directly used in the system or it may undergo fine-tuning. By *Performing Fine-Tuning*, the stakeholders tailor the selected pre-trained model to the specific defined requirements by additional training on the extracted data, adjusting parameters to optimize for particular types of tasks, or even modifying the model architecture to better suit specific tasks. In case of deciding to *Perform Fine-Tuning*, the

required data undergoes the steps of extraction to validation similar to *Performing Offline ML Experimentation*.

If large language models (LLMs) are employed during this activity, the stakeholders may also *Perform Prompt Engineering* and *Perform In-Context Learning*. By *Performing Prompt Engineering*, they iteratively design and optimize “prompts”, i.e., the textual instructions for the LLM, to guide model behavior for their specific requirements and tasks. By *Performing In-Context Learning*, they provide the model with input-output examples for a specific task, e.g., translation, classification, or arithmetic, so that the model can use those examples as context to infer the task pattern and generate the correct output for new inputs. These steps are therefore specific to LLMops and are usually not needed in conventional MLOps.

Key insights from RQ2.1

1. While several different instantiations of the MLOps process exist in literature, the steps can be grouped into 8 categories covering both ML-specific aspects (data engineering, ML training including hyperparameter optimization, model versioning and deployment, inference) and system aspects typical of any software product (requirements, design, implementation, testing, CI/CD).
2. The way the MLOps process is instantiated is governed by important decisions, including whether to consider online re-training, based on what to trigger such re-training, whether newly trained ML models are automatically deployed, and whether an Inference Engine or an Inference Service is used.
3. When pre-trained (including foundation) models are used, the MLOps activity of Offline ML Experimentation needs to be adapted to account for the selection of such models and their potential fine-tuning. In the emerging case of LLMs, these steps are typically followed by prompt engineering and in-context learning.

4.4. Iterative nature via feedback loops (RQ2.2)

The process view in Fig. 8 is visualized linearly. However, the process of MLOps systems is intuitively highly non-linear. To visualize the iterative non-linear aspect, this process view is complemented by *Feedback Loops (FBLs)*, which enable the continuous improvement of ML models in the end-to-end MLOps process. In the following, we describe the 13 synthesized FBLs explicitly mentioned in the 93 primary studies. The FBLs are also visualized in Fig. 10.

We describe three general categories, based on the starting point of the FBL:

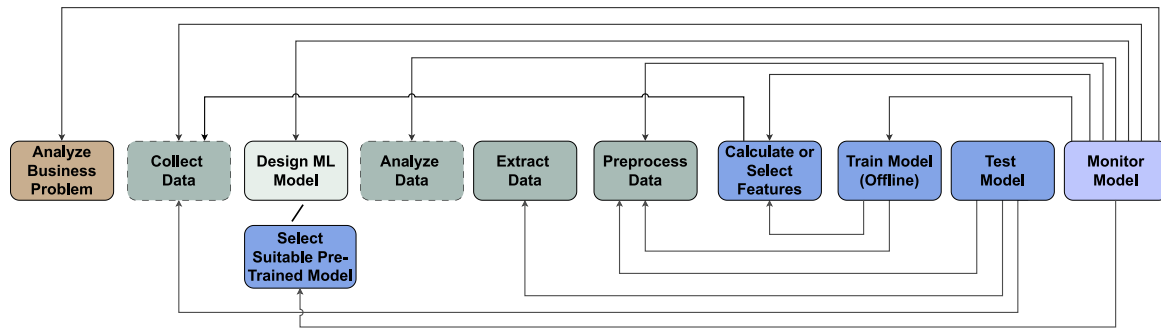


Fig. 10. Feedback loops in an MLOps process.

1. **FBLs from Monitor the Model:** This is the most common and emphasized feedback loop (20 explicit mentions among the papers). As part of the continuous action *Monitor the Model*, the process of an MLOps system can go back to one of the following actions:

- *Analyze Business Problem*: This FBL signifies the comprehensive restart of the entire MLOps process, requiring that all actions are revisited from the initial stage.
- *Collect Data* signifies the essence of access to additional or more relevant data than is already available in the MLOps system. Naturally, the subsequent actions are also revisited.
- *Design ML Model* or *Select Suitable Pre-Trained Model*: This FBL implies the revision of the *Perform ML Experimentation* activity, hence the design, development, and training of new ML models. In case of using pre-trained models instead of training models from scratch, the FBL implies the revision of the choice in selecting the suitable pre-trained model.
- *Analyze Data*: By returning to this action, the data scientist and the data engineer analyze the available data, and based on their findings, determine the appropriate next actions. These actions may include collecting additional data, extracting new or more relevant data, applying different preprocessing techniques, or calculating new features based on the data.
- *Preprocess Data* performs more or different preprocessing steps on the available data.
- *Calculate or Select Features* is revised to retrain models with different features.
- *Train Model* (in Offline Experimentation) retrains the models by adjusting hyperparameters and reusing the same data and the code.

2. **FBLs from Test Model:** After the trained ML model is tested on the specified metrics, the process may go back to one of the following actions of the *Data Engineering* category:

- *Collect Data* to collect additional or different types of data.
- *Extract Data* to extract additional or different data.
- *Preprocess Data* to perform preprocessing techniques on data, e.g., normalizing features.

Depending on the test results, the data scientist can decide on which action to revisit in the process.

3. **FBLs from Train Model in the 'Offline ML Experimentation' activity:** From this action, the MLOps process can go back either to *Preprocess Data* or to *Calculate or Select Features* to apply different preprocessing techniques or select different sets of features to retrain the ML models.

4. **FBL from Calculate or Select Features:** From this action, the MLOps process may go back to *Collect Data*. If the features do not satisfy the stakeholders, they may decide to collect more data, or augment and generate data.

It is important to note that the FBLs mentioned here are the most commonly performed ones and are derived from the reviewed papers. However, other steps in the process may also be revisited at any time. For example, stakeholders may choose to collect and label additional data concurrently at any stage for future use.

Key insights from RQ2.2

1. Our study reveals 13 feedback loops in the MLOps process that are explicitly mentioned in the literature. This underlines the importance of feedback loops in maintaining and improving the performance of ML models in an MLOps system.
2. The most common feedback loop returns the control of the process from *Monitor Model* to *Analyze Business Problem*, i.e., the restart of the whole process.

4.5. Stakeholder roles and their responsibilities (RQ2.3)

As shown in Table 4, each action is carried out by an individual, a group of stakeholders, or the system (in the automated actions). The MLOps process as an interdisciplinary team effort requires effective collaboration across various roles. In total, we identify 9 main human stakeholder roles and 5 role specializations; in addition, we consider the automated actions that are performed by the system. In the following, we describe these stakeholder roles (which can overlap), together with their aliases, specializations, and responsibilities within the MLOps process. The presence of specialized roles depends on the composition of the MLOps team and the organization's maturity level [43]. In the absence of specialized roles, tasks can be performed by their parent roles, as detailed below.

- **Domain Expert**, also known as *Business Stakeholder*, *Process Engineer*
 - Defines and analyzes the business problem and provides requirements to be solved by ML-based solutions.
 - Assesses the quality and usability of gathered requirements together with the *data scientist*.
 - Provides metrics based on the domain to evaluate the ML models.
 - Provides insights to *data scientists* in the steps *Test Model* and *Monitor Model*.
 - Consumes the results of the ML models and presents the metrics achieved from ML models to the interested parties.

- Labels the collected data to provide ground truth for training the ML models.
 - **Business Analyst**; Specialization: *ML Owner*
 - Studies the business goal feasibility and translates the business goal into tasks that can be solved by ML-based solutions.
 - Conveys ML requirements to business stakeholders during development and deployment phases.
 - The *ML owner*, as a specialization of this role, provides the other ML actors (stakeholders) with the required tools, materials, and resources. In the absence of an ML owner, the *business analyst* provides the tools, materials, and resources.
 - **Solution Architect**, also known as *System Architect*, *IT Architect*; Specialization: *ML Architect*
 - Designs the overall MLOps system architecture and determines the technologies to be used in the system.
 - The *ML architect*, as a specialization of this role, translates the technical requirements of the defined tasks into architecture components for the ML parts. The ML architect also ensures that the MLOps system can address the scale of the identified requirements regarding predictions. In the absence of an ML architect, the *solution architect* performs these tasks in close collaboration with the *data scientist* to ensure alignment with the ML requirements.
 - **Data Scientist**, also known as *ML Expert*; Specialization: *ML Model Tester*
 - Analyzes the data and evaluates the data with respect to its quality and quantity.
 - Develops, trains, evaluates, and releases operational ML models that address the identified business goal.
 - In the absence of the *business analyst*, the *data scientist* also translates the business problem into an ML problem.
 - In the absence of a *DevOps engineer*, the *data scientist* can also perform the task of continuous monitoring of ML models and is notified by the monitoring component in case of model performance degradation based on defined thresholds.
 - The *ML model tester*, as a specialization of this role, performs model assessment and validates models before releasing them to the production environment. In the absence of this role, the *data scientist* performs the model validation and release.
 - **Software Engineer**, also known as *Application Developer*
 - Implements the non-ML parts of the MLOps system that consume model predictions.
 - Integrates the trained and validated ML models into the respective components and ensures the operational stability of the complete system.
 - Ensures that design and code (also of ML parts) adhere to widely accepted best practices and patterns.
 - Guides the *DevOps engineer* in developing platform-related components.
 - **Data Engineer**; Specialization: *Data Steward*
 - Creates and manages data and feature engineering pipelines.
 - Ensures proper data extraction and storage in *Data Repository* and *Feature Store*.
 - Transforms raw data to clean, structured data that can be used in training the ML models.
 - *Data Steward*, as a specialized role, preserves overall dataset quality, supervises data in different stages, prevents leakage of sensitive data, and provisions access to certain relevant data sources for other stakeholders. In the absence of this role, the *data engineer* takes over these tasks.
 - **DevOps Engineer**, also known as *Operations/Ops Engineer*; specialization: *MLOps Engineer*
 - Implements, tests, and deploys the platform-related components.
 - Ensures proper model deployment to production, CI/CD automation, and monitoring.
 - Is responsible for the continuous monitoring of ML models and is notified by monitoring components in case of model performance degradation based on defined thresholds.
 - The *MLOps engineer*, as a specialized role with cross-domain knowledge, supervises and manages the MLOps infrastructure, i.e., infrastructure components for the ML parts. In the absence of this role, the *DevOps engineer* takes over this task.
 - **ML Engineer**
 - Creates the code for the automated ML training pipeline and saves it in the *Code Repository*.
 - Leads the development of ML tasks and communicates and manages the cooperation between the different ML actors.
 - This role has a cross-domain portfolio and manages the automated ML training pipeline and model deployment to production.
 - In the absence of a *DevOps engineer*, this role can also be responsible for the continuous monitoring of ML models and is notified by monitoring components in case of model performance degradation based on defined thresholds.
 - **End-User**, also known as *User*
 - Consumes the results/predictions of the ML models and acts upon these results.
 - Can provide feedback on the performance and effectiveness of the ML models.
 - **System**
 - Performs the automated actions or activities within the MLOps process.
 - The level of automation depends on the architectural design decisions and requirements of the system, e.g., making decisions based on ML model predictions during inference may be performed by the *System* automatically, or by the *End-User* manually.
 - The actions and activities performed by the *System* itself are referred to as the ‘automated actions’.
- To better depict the responsibilities of each role and the automated actions in the process of an MLOps system, we provide the stakeholder-action mapping in Fig. 11. As depicted in the figure, the data scientist performs the majority of the tasks, while some other roles perform one or two tasks in the complete process.

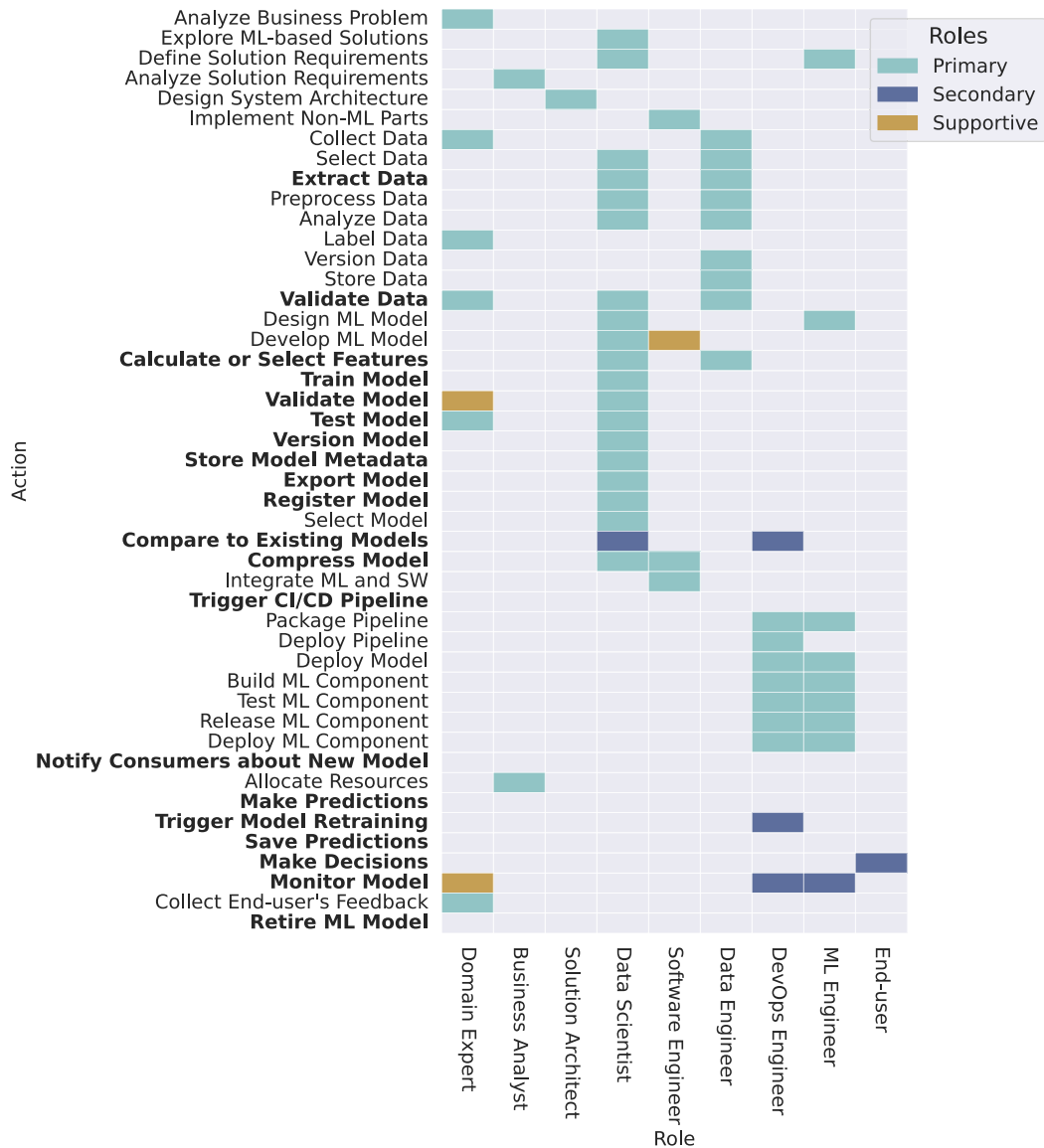


Fig. 11. The roles mapped to the actions. The action names in bold indicate the automated actions (performed by the system).

Key insights from RQ2.3

1. Our study reveals 9 main stakeholder roles involved in a successful MLOps process. With such a high number of roles, effective collaboration is paramount.
2. In certain steps of the process, responsibility for a task does not rest with a single stakeholder; rather, multiple stakeholders collaborate or are consulted for the successful execution of these steps. For example, while monitoring the models, the domain expert plays a supportive role and helps the responsible stakeholders by providing his or her insights regarding the interpretation of the monitoring results.
3. Even though some steps in the process are fully automated, certain stakeholders are responsible for these steps and observe their status and progress, e.g., ML engineer and DevOps engineer supervise *Monitoring Models* or are notified by the system based on certain thresholds.

5. Discussion

The synthesized UML component and activity diagrams plus the tool-entity and stakeholder-step mappings can already serve as valuable references for practitioners and researchers who aim to design or enhance MLOps systems. In addition, in this section, we discuss several noteworthy points related to (1) the representation of MLOps architectures in literature, (2) important decisions we had to make in this study, and (3) opportunities for future research in the area of MLOps architectures.

5.1. Representation of MLOps architectures in literature

Complexity and mixed views in architecture diagrams. A majority of the analyzed figures and descriptions of MLOps architectures are a combination of architectural structure, process, and stakeholder roles, thereby combining several different architectural views and concerns in a single diagram. Even though this approach is a common practice and makes the provided information more compact, the increased complexity reduces the clarity of the provided architectures. The complexity also affects our synthesis. When reporting the number of mentions of each structural entity and process step in Tables 3 and

4, we occasionally lose information about the number of occurrences of core entities in an MLOps system. We always extract an entity for only one of the two views, i.e., this problem occurs when certain structural entities are referenced as process steps in the mixed-view architecture, or conversely, when process steps are referred to as structural entities. Examples include the *ML Experiment Pipeline* component and *Train Model* step.

Non-standard notations to represent architectures. Almost all of the analyzed figures use informal box-and-line diagrams as a notation. Among the 93 primary studies,³ only one paper (S23) uses a standard notation, the Fundamental Modelling Concepts (FMC) [143]. Combined with the complexity of the mixed views and following the results of Warnett and Zdun [39], the understandability of these MLOps architecture representations is substantially impacted.

Level of abstraction in architectures. Among the extracted figures and descriptions of the papers, the level of provided details and abstraction varied over a large spectrum. During the study, we extracted and synthesized data from figures only representing as few as six high-level architecture components (S10, S43) or four high-level process actions (S9) to complex “combined figures” with over 40 concrete entities, including components, actions, and stakeholder roles (S7). The different levels of abstraction and multitude of dependencies between components in the structural view allow many fine-grained possibilities for variations in component dependencies and interface directions. In the process view, the levels of abstraction also allow a myriad of possibilities for sequences of activities. For simplicity, we modeled only the most common dependencies in the diagrams from a detailed viewpoint.

Tools in place of architectural entities. In addition to the mixed architectural views and different levels of abstractions, the extracted figures in some papers also included a mixture of architectural entities and the employed tools as a stand-alone entity (see, e.g., S31). This mix of generalized components and concrete implementation technologies in the views makes the architectural understanding and comparison difficult.

Inconsistently named architectural entities. The terminology for the same individual architecture components or process steps could vary substantially among the papers. We identified two types of inconsistent naming: (a) some papers use common entity names that imply different responsibilities, e.g., using “Feature Store” for a component that provides the training data for the ML training pipeline instead of “Data(set) Store”, using “Artifact Store” instead of “ML Model Store”, or using “Pattern Analysis” instead of “Data Analysis”, and (b) other papers use unique names to represent common ML-based software components, e.g., “knowledge base manager” for a component that stores and versions the trained ML models (*Model Repository* in our synthesis), “Training Data Creation” for an action that “Labels Data”, or “Model Analysis” for the action of “Evaluating Models”. These inconsistencies highlight why communication and collaboration in MLOps projects is often difficult.

Specific terms used interchangeably to represent different architectural entities. Besides the inconsistency in naming the architectural entities, we observe in some papers that specific terms are used interchangeably. These terms include:

- *Data Ingestion* refers to both actions *Collect Data* and *Extract Data*, but the former means collecting raw data from sources and the latter means extracting or fetching already saved data from a *Data Repository*.
- *Model Evaluation*, *Model Validation*, *Model Testing*, and *Model Verification* are used interchangeably to refer to actions of ‘*Evaluating the trained model performance on the validation data*’ and ‘*Testing the model with respect to not only performance but different metrics*’. In

our synthesis, we name the former as *Validate Model* and define it as the iterative evaluation of an ML model with returning to *Train Model* until the expected performance is reached, while the latter is named *Test Model* and it refers to final validation of the trained model and assessing it with respect to different metrics.

- *Model Building* refers to both *Design ML Model* and *Train Model*. *Build* as a single term is also observed to refer to *Train Model* and *Build ML Component*.

To address the discrepancy in naming the architectural entities, we provide component and action aliases as an indication of which components and actions are known similarly among the community and which are addressed by a variety of names.

End-to-end tools only mentioned for a certain set of components. A tool-related observation is that most of the end-to-end MLOps tools like MLflow, Kubeflow, or AWS SageMaker are recommended only for a subset of components, rather than for the entire workflow. For example, in the reviewed papers, MLflow is mentioned only for components in the *Storage and Versioning*, *Inference*, and *Infrastructure and Supporting Services* among our categories. Interestingly, this tool is not mentioned for any component in the *ML Training* category.

MLOps representation in gray literature. In this study, we analyze MLOps architectures in white literature, i.e., scientific and peer-reviewed sources. To complement our synthesis, we compare our results with four reputable practitioner-oriented sources drawn from industry practice, namely two Google sources [144,145], one Microsoft source [146], and one Databricks source [147]. While these gray-literature sources predominantly emphasize technical, practical, and implementation concerns, our analysis focuses on the structural and process views of MLOps architecture.

We first compare our findings with the representation in [144], where MLOps is defined as “a set of standardized processes and technology capabilities for building, deploying, and operationalizing ML systems rapidly and reliably”. Similar to most of the white-literature sources that we reviewed, this document presents MLOps as a combination of processes and technical capabilities within a mixed-view architecture. Their “technical capabilities” correspond closely to the structural entities in our structural view, while their described “processes” align with our process steps. In contrast to this source, we decompose both views at a finer, atomic level and explicitly distinguish the tasks of each architectural entity. Furthermore, whereas the Google source associates stakeholders broadly with each process, our architecture details stakeholder tasks and responsibilities, relating them precisely to individual process steps.

The second and third sources [145,146] conceptualize MLOps from a maturity-level perspective, primarily based on the level of automation of the system components. In the second Google source [145], three maturity levels are defined according to the degree of automation in ML and CI/CD pipelines. This source visualizes the ML training and deployment process, whereas our synthesis encompasses the end-to-end MLOps system—starting with analyzing the business problem and data collection, and extending to serving predictions and model retirement decisions in inference. Moreover, while Google builds maturity levels around pipeline automation, we instead provide architectural variants based on the presence or absence of different architectural entities. Similarly, Microsoft [146] defines five maturity levels grounded in the technical capabilities of the MLOps system. This source conceives MLOps primarily as a process and organizational culture, whereas our work adds a clear architectural viewpoint.

The reference architecture by Databricks [147] emphasizes the implementation perspective, with strong attention to distinguishing development, staging, and production environments. The authors highlight that stakeholders should explicitly label code and data according to these environments and handle them accordingly. In their reference architecture, the authors also provide detailing practices such as access control for different stakeholder groups. In contrast, as mentioned

³ The complete list of primary studies can be found in Table 2.

earlier, our architecture concentrates on structural and process views, leaving implementation practices as future work. Additionally, while these practitioner sources mention stakeholders only briefly as part of process steps, our architecture explicitly specifies the tasks and responsibilities of each stakeholder. This architecture also presents a fully automated MLOps process, whereas our work offers architectural variants based on whether individual architectural entities (or groups of entities) are present. This source also provides best practices for LLM-specific tasks (LLMOps), treating LLMOps as an *adaptation of MLOps*—a position we share, as we also define LLMOps as a variant of MLOps with additional architectural entities tailored to LLM-specific tasks. One such set of entities described by Databricks but absent from our architecture is RAG (Retrieval-Augmented Generation), which provides mechanisms for leveraging external data in LLMs. The authors present RAG alongside other data-leveraging approaches, such as prompt engineering and fine-tuning, and note that these methods may be combined depending on the use-case requirements. Finally, Databricks also emphasizes the increased importance of human feedback in LLMOps relative to traditional MLOps, arguing that while human feedback may be optional in classical ML, it is essential in most LLM applications. Human feedback must therefore be managed as a first-class data type, ideally integrated into near-real-time monitoring pipelines.

5.2. Important decisions of this study

Domain-specific architecture components and process actions.

In more than half of the papers, the provided architectures are domain-specific and therefore contain components or actions that are only situationally applicable or focused on a specific application scenario. For example, the presence of “additive manufacturing” components in S36, “Blockchain”-related components in S40, and actions related to the process of “Image Analysis” in S33. In the analysis, we generalize these components and actions to a relevant category or discard the ones that represent very specialized entities that are of no generalizable value outside their domain.

Core and add-on architectural entities and the MLOps system design decisions. As described in Section 3, we synthesize an architectural entity as a core or an add-on entity based on different factors in our observation of the state of the art. However, the presence of these entities in an MLOps system depends on many other factors that are not easily observable in a UML diagram. For example, the presence of some entities highly depends on other entities, the conditions of the system, and the granularity of the components (e.g., if the MLOps stakeholders decide to use one centralized database for data, models, metadata, etc., the other repositories in our architecture would be optional). This limited presentation can be complemented with MLOps practices and guidelines, which we are planning to provide in a subsequent study.

Implicit architecture components and process actions. As a result of the high level of abstraction in some architecture figures and descriptions, some component responsibilities and process actions are left implicit. To have a rigorous fine-grained architecture, we divided the responsibilities of the abstract architectures. For instance, in many papers, aliases of *Train Model* refer to the iterative process of *Train Model* and *Validate Model*. Moreover, the action of *Select Data* remains implicit in most of the architectures that we reviewed. For the comprehensiveness of the architecture and granularity of the actions, we add this action to our process view as a core action.

Implicit stakeholder responsibilities. Similar to the implicit components and actions, we observe that certain stakeholder roles are not explicitly mapped to their respective process actions. However, their responsibilities become clear from the task descriptions provided in some papers. Through our analysis, these roles have been synthesized and mapped to the corresponding actions.

Collection and labeling of data during inference. In the process view of our proposed architecture, *Collect Data* and *Label Data* are visualized similarly to classic ML training pipelines, i.e., before

performing offline ML experimentation. However, as observed in some papers (e.g., S18 and S29), the provided data by users in inference may be collected and labeled to serve as new training data for the offline ML experimentation or online ML training.

Transitioning models to a staging environment before deployment to production. When a model is trained and ready to be deployed to production, it is best practice to first deploy the model to a staging environment. However, we are not depicting this case explicitly in our diagrams, as we only observe this explicitly in one of the reviewed papers (R34), and implicitly in a few papers.

Saving the metadata related to all the models vs. the selected models only. When different models are trained in the *Perform ML Experimentation*, either all the trained models and related metadata or only the ones selected by the *Select Model* are stored. It is an MLOps practice that depends on the resource availability and storage capacity of the system. We only visualize the former case in our process view.

5.3. Opportunities for future research

Feedback Loops. A total of 21 out of 93 papers explicitly model FBLs in the analyzed figures. Since the MLOps process is inherently iterative, we expected to find a higher percentage. Here we list the FBLs that we believe should also be considered in an MLOps system but are not a result of our data extraction:

- FBL from *Validate Model* to *Extract Data* or subsequent actions. In model evaluation, the data scientist may not only need to return to adjusting the hyperparameters for retraining, but they may need more or different sets of data for which they need to revisit *Extract Data*.
- FBL from *Validate Model* to *Design ML Model*. Using their expertise, the data scientist may decide to select or design a completely new model and train it considering the available data.
- FBL from *Test Model* to *Design ML Model*. Similarly to the case above, the data scientist may decide to design a completely new model and train it after observing the validation results.

A follow-up study, potentially involving interviews and/or gray literature, could focus on further clarifying the feedback loops in MLOps architectures since they are key to smooth system operation and collaboration between different stakeholders.

Cross-cutting concerns in the overall system architecture. As a result of our mapping study, we do not identify cross-cutting concerns such as security, compliance, trustworthiness, and traceability, among others, as standalone architectural entities. Instead, these concerns are generally embedded implicitly within broader architectural descriptions. This is consistent with the role these aspects play in real-world ML and LLM systems: rather than functioning as isolated units, they typically operate as cross-cutting operational practices that span multiple architectural entities, including data engineering, model training, deployment, monitoring, and inference. We therefore treat these concerns as practices related to architecting MLOps systems and consider them a direction for future research. As an important example of cross-cutting concerns, we explicitly address *failure management* in the following.

Failure management in MLOps systems. Failure management is considered to be an important aspect of MLOps systems, ensuring the reliability and robustness of deployed ML models [148,149]. In an MLOps system, we recognize failure management as a key function of the *Runtime Model Monitor* component, which differentiates these systems from traditional ML-enabled systems. Since our current observations capture failure management within the broader architecture, primarily as an operational practice that spans multiple architectural entities, we identify the value of examining it in greater detail. When decomposed into the three core phases of (1) failure detection, (2) failure diagnosis, and (3) recovery mechanisms, our findings indicate that each phase is indirectly supported within the proposed MLOps architecture through the following mechanisms:

1. **Failure Detection** is achieved through continuous monitoring of both system infrastructure and model performance. This includes drift detection, threshold-based alerting, and other metrics-driven techniques, as described in Section 4. Generally, the system can optionally *Trigger Model Retraining* or notify relevant stakeholders when degradation is detected. The *MLOps User Interaction Manager* provides dashboards that help stakeholders interpret the results and respond to these alerts.
2. **Failure Diagnosis** focuses on investigating root causes of detected failures or performance degradations. This partly happens in the *Analyze Data* step via analyzing data distributions, but also involves reviewing logs and analyzing system-level alongside model performance metrics.
3. **Recovery Mechanisms** involve a range of strategies, including automated rollback to a previous model version, retraining the models using updated data, or switching to an alternative model in inference. Canary testing, A/B testing, and shadow deployment are commonly mentioned techniques for this phase.

These mechanisms are embedded in different architectural components, though most primary studies that we reviewed did not address them or addressed them only implicitly. Building on these insights, we see an opportunity for future work to provide a more comprehensive treatment of failure management, including concrete implementation strategies and operational best practices, to strengthen the practical guidance for architecting MLOps systems and complement our current architectural view.

Tools mapped to process steps in the reviewed papers. In this paper, we provide a mapping between the tools and the architectural entities that they support. However, during the data synthesis, we observed that the trend in the newer publications is towards mentioning the tools that are used by the stakeholders in each process step. This is a viewpoint that could be helpful for practitioners. However, we consider it outside the scope of this study and only synthesize tool-component mapping, leaving the tool-step mapping as future work.

Applicability of MLOps architectures for LLM-specific cases and LLMops. Based on our architecture and process analyses, we conclude that LLMops still relies on the majority of MLOps entities. Despite some small implementation differences that are mostly due to reusing pre-trained models, the context of the provided architecture does not change substantially. From an architectural viewpoint, we do not observe major characteristics that would require a fundamentally different perspective for LLMops. Nonetheless, a few components like the *Fine-Tuning Module* or process steps like *Perform Prompt Engineering* are very closely aligned with LLMops and lose their importance in the absence of pre-trained LLMs. We also suspect that current MLOps and LLMops literature does not yet include some additional components that are often used when training and deploying LLMs. For example, it is common practice to fine-tune deployed LLMs guided by reinforcement learning from human feedback gathered from a production system [150]. Such practice clearly requires additional components currently not described in the LLMops literature. Given the rapid increase in the importance of LLMs and LLMops, this may change in the next few years, motivating a follow-up study to ours. However, for now, the overall small number of LLM-specific entities supports the idea of LLMops being a specialization of MLOps and not a fundamentally different paradigm, at least from an architectural perspective.

6. Threats to validity

Following the categorization by Wohlin et al. [151] and the checklist by Ampatzoglou [48], we outline the threats that may have affected the validity of our research and outline the actions that we take to mitigate the threats.

Threats to *internal validity* undermine the conclusion about a possible causal relationship between the study and the outcome [151]. To

mitigate this threat, we assign two researchers for the study selection and data extraction phases, who perform the selection and extraction independently, and then three researchers discuss the results in consensus meetings. Another possible threat to internal validity in this study stems from the selection of papers from various domains. Varying author expertise across fields, particularly outside ML-enabled systems, may influence the accuracy of the architecture and process descriptions in papers, which influences our data extraction. To mitigate this threat, we have mapped every component and activity name and description to the authors' intentions, e.g., in some papers authors refer to a dataset repository as a feature store. Additionally, since we do not apply a formal quality assessment, our primary studies include papers of varying rigor. While this approach aligns with the goal of systematic mapping studies – capturing the breadth of existing work – it may affect how the extracted data and interpretations should be used, e.g., they may represent more what exists rather than what is best. To mitigate this threat, we compare our synthesized results to several reputable gray literature sources.

External validity is concerned with the generalization extent of the findings of the study [151]. A possible threat to the external validity of our study is the selection of the papers among the peer-reviewed resources, which may limit our initial set of papers. This is a research design decision to ensure the reliability of the selected papers. We mitigate this threat by applying our search query on Google Scholar as a meta-engine, which results in papers from different venues, as well as applying one round of bidirectional snowballing. However, since we focus on peer-reviewed scientific papers, we might miss works from the many practitioners that certainly focus on this very popular topic. This may introduce a threat to the generalizability of the results. However, among the reviewed papers, 12/93 papers were authored by industrial practitioners and 23/93 as a collaboration between industry and academia, which provides a decent amount of practitioner insights. Using Google Scholar as the starting point for collecting papers introduces the possibility that the results may not be fully repeatable, as Google Scholar rankings can vary based on factors such as search location, query history, and personalization. However, because our search strategy is title-focused and we clearly specify the time span of the search, we expect a substantial degree of repeatability. In addition, we provide the full set of retrieved search results both in the paper and in the replication package [141], enabling readers to verify and reproduce our dataset.

Reliability is concerned with the extent to which the data and the analysis are dependent on the specific researchers [48,152]. A possible threat in our study concerning reliability can be the authors' bias in synthesizing data due to the nature of the extracted data and the different levels of abstraction of the architecture in the papers. To mitigate this threat, three authors participate in analyzing and synthesizing the data.

7. Conclusion

Based on a systematic mapping study (SMS) with 93 scientific papers, we provide an architecture of MLOps systems using a structural and a process view. Towards providing the structural view, we synthesized architecture components, their dependencies, and tools to implement them. Similarly, for the process view, we synthesized actions and activities, their sequence and feedback loops, and the stakeholders of the MLOps systems in this process. Furthermore, we discuss several synthesized architecture variants that emerge from the literature and can be used by MLOps researchers and practitioners to choose among and design their own tailored architectures. Our results contribute to understanding the architectural aspects of MLOps systems and potentially support communication in this complex and still maturing field.

Having provided a conceptualization of MLOps architectures in this study, we identified that practitioners could be best supported if

we could clearly link MLOps architecture variants to functional and quality requirements so that they can easily choose the variant that best suits their needs. To this aim, a synthesis of common stakeholder requirements would be needed, which we have not identified in the literature for MLOps systems so far. Hence, some lines of work for future directions for interested researchers and practitioners are (i) to synthesize common stakeholder requirements in MLOps systems, (ii) to provide a mapping of the elicited requirements to architectural entities and the provided architecture variants, and (iii) to evaluate the conceptualized architecture based on practitioner views of usability and applicability of the provided architecture to their needs. The latter one helps limit the gap between research and practice.

Our future work plan is to provide a reference architecture for MLOps systems. To this end, synthesizing architectural decisions, best practices, and antipatterns in this domain, which we plan to provide as guidelines in our envisioned reference architecture, may further support practitioners. Furthermore, the envisioned guidelines and the mapping of the stakeholder requirements can be used as a roadmap by practitioners to select the most suitable architecture variants and instantiate the most suitable architectures from the provided blueprints. To allow such endeavors and to increase transparency, we share our research artifacts on Zenodo [141].

CRedit authorship contribution statement

Faezeh Amou Najafabadi: Writing – review & editing, Writing – original draft, Visualization, Methodology, Formal analysis, Data curation, Conceptualization. **Justus Bogner:** Writing – review & editing, Writing – original draft, Methodology, Formal analysis, Data curation, Conceptualization. **Ilias Gerostathopoulos:** Writing – review & editing, Writing – original draft, Supervision, Methodology, Formal analysis, Data curation, Conceptualization. **Patricia Lago:** Writing – review & editing, Supervision, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research is supported by ExtremeXP, a project co-funded by the European Union Horizon Programme under Grant Agreement No. 101093164.

Data availability

All the data is shared in the replication package, and the link is provided in the manuscript.

References

- [1] D. Kreuzberger, N. Kühl, S. Hirschl, Machine learning operations (MLOps): Overview, definition, and architecture, *IEEE Access* 11 (2023) 31866–31879, <http://dx.doi.org/10.1109/ACCESS.2023.3262138>.
- [2] M.M. John, H.H. Olsson, J. Bosch, Towards MLOps: A framework and maturity model, in: 2021 47th Euromicro Conference on Software Engineering and Advanced Applications (SEAA), IEEE, 2021, pp. 1–8, <http://dx.doi.org/10.1109/SEAA53835.2021.00050>.
- [3] L. Bass, I. Weber, L. Zhu, *DevOps: A Software Architect's Perspective*, first ed., Addison-Wesley Professional, 2015.
- [4] M. Treveil, N. Omont, C. Stenac, K. Lefevre, D. Phan, J. Zentici, A. Lavoillotte, M. Miyazaki, L. Heidmann, *Introducing MLOps*, O'Reilly Media, Inc, 2020.
- [5] A. Kolar Narayanappa, C. Amrit, An analysis of the barriers preventing the implementation of MLOps, in: *International Working Conference on Transfer and Diffusion of IT*, Springer, 2023, pp. 101–114, http://dx.doi.org/10.1007/978-3-031-50188-3_10.
- [6] S. Idowu, D. Strüder, T. Berger, Asset management in machine learning: a survey, in: *Proceedings of the 43rd International Conference on Software Engineering: Software Engineering in Practice*, in: ICSE-SEIP '21, IEEE Press, 2021, pp. 51–60, <http://dx.doi.org/10.1109/ICSE-SEIP52600.2021.00014>.
- [7] S.J. Warnett, U. Zdun, Architectural design decisions for machine learning deployment, in: 2022 IEEE 19th International Conference on Software Architecture (ICSA), 2022, pp. 90–100, <http://dx.doi.org/10.1109/ICSA53651.2022.00017>.
- [8] R. Nazir, A. Bucaioni, P. Pelliccione, Architecting ML-enabled systems: Challenges, best practices, and design decisions, *J. Syst. Softw.* 207 (2024) 111860, <http://dx.doi.org/10.1016/j.jss.2023.111860>, URL: <https://linkinghub.elsevier.com/retrieve/pii/S0164121223002558>.
- [9] E.Y. Nakagawa, P.O. Antonino, *Reference Architectures for Critical Domains: Industrial Uses and Impacts*, Springer Nature, 2023, <http://dx.doi.org/10.1007/978-3-031-16957-1>.
- [10] F. Amou Najafabadi, J. Bogner, I. Gerostathopoulos, P. Lago, An analysis of MLOps architectures: A systematic mapping study, in: *Proceedings of the 18th European Conference on Software Architecture*, Springer, 2024, pp. 69–85, http://dx.doi.org/10.1007/978-3-031-70797-1_5.
- [11] W.P. Luz, G. Pinto, R. Bonifácio, Building a collaborative culture: a grounded theory of well succeeded DevOps adoption in practice, in: *Proceedings of the 12th Acm/Ieee International Symposium on Empirical Software Engineering and Measurement*, ESEM, 2018, pp. 1–10, <http://dx.doi.org/10.1145/3239235.3240299>.
- [12] A. Hemon, B. Lyonnet, F. Rowe, B. Fitzgerald, From Agile to DevOps: Smart skills and collaborations, *Inf. Syst. Front.* 22 (4) (2020) 927–945, <http://dx.doi.org/10.1007/s10796-019-09905-1>.
- [13] S. Badshah, A.A. Khan, B. Khan, Towards process improvement in DevOps: a systematic literature review, in: *Proceedings of the 24th International Conference on Evaluation and Assessment in Software Engineering*, EASE, 2020, pp. 427–433, <http://dx.doi.org/10.1145/3383219.3383280>.
- [14] A. Katal, V. Bajoria, S. Dahiya, DevOps: Bridging the gap between development and operations, in: 2019 3rd International Conference on Computing Methodologies and Communication, ICCMC, 2019, pp. 1–7, <http://dx.doi.org/10.1109/ICCMC.2019.8819631>.
- [15] J. Bosch, Continuous software engineering: An introduction, in: *Continuous Software Engineering*, Springer, 2014, pp. 3–13, http://dx.doi.org/10.1007/978-3-319-11283-1_1.
- [16] B. Fitzgerald, K.-J. Stol, Continuous software engineering: A roadmap and agenda, *J. Syst. Softw.* 123 (2017) 176–189, <http://dx.doi.org/10.1016/j.jss.2015.06.063>.
- [17] G.A. Lewis, I. Ozkaya, X. Xu, Software architecture challenges for ML systems, in: 2021 IEEE International Conference on Software Maintenance and Evolution, ICSME, IEEE, 2021, pp. 634–638, <http://dx.doi.org/10.1109/ICSME52107.2021.00071>.
- [18] S. Amershi, A. Begel, C. Bird, R. Deline, H. Gall, E. Kamar, N. Nagappan, B. Nushi, T. Zimmermann, *Software Engineering for Machine Learning: A Case Study*, 2019, pp. 291–300, <http://dx.doi.org/10.1109/ICSE-SEIP.2019.00042>.
- [19] R. Subramanya, S. Sierla, V. Vyatkin, From DevOps to MLOps: Overview and application to electricity market forecasting, *Appl. Sci.* 12 (19) (2022) 9851, <http://dx.doi.org/10.3390/app12199851>.
- [20] S. Pahune, Z. Akhtar, Transitioning from MLOps to LLMops: Navigating the unique challenges of Large Language Models, *Information* 16 (2) (2025) <http://dx.doi.org/10.3390/info16020087>.
- [21] J. Diaz-De-Arcaya, J. López-De-Armentia, R. Miñón, I.L. Ojanguren, A.I. Torre-Bastida, Large language model operations (LLMOps): Definition, challenges, and lifecycle management, in: 2024 9th International Conference on Smart and Sustainable Technologies (SpliTech), IEEE, 2024, pp. 1–4.
- [22] M. Sinha, S. Menon, R. Sagar, LLMops: Definitions, framework and best practices, in: 2024 International Conference on Electrical, Computer and Energy Technologies (ICECET), 2024, pp. 1–6, <http://dx.doi.org/10.1109/ICECET61485.2024.10698359>.
- [23] R. Shan, T. Shan, Enterprise LLMops: Advancing large language models operations practice, in: 2024 IEEE Cloud Summit, 2024, pp. 143–148, <http://dx.doi.org/10.1109/Cloud-Summit61220.2024.00030>.
- [24] IBM, What Are Large Language Model Operations (LLMOps)? | IBM, 2023, URL: <https://www.ibm.com/think/topics/llmops>.
- [25] Databricks, What Is LLMops? 2023, URL: <https://www.databricks.com/glossary/llmops>.
- [26] Google, LLMops: What it is and how it works, 2025, URL: <https://cloud.google.com/discover/what-is-llmops>. (Accessed 12 May 2024),
- [27] Weights and Biases, Understanding LLMops: Large language model operations, 2023, URL: <https://wandb.ai/site/articles/understanding-llmops-large-language-model-operations/>,
- [28] Valohai, LLMops for large language models, 2023, URL: <https://valohai.com/blog/llmops/>.
- [29] Microsoft, An introduction to LLMops: Operationalizing and managing large language models using Azure ML, 2023, URL: <https://techcommunity.microsoft.com/blog/machinelearningblog/an-introduction-to-llmops-operationalizing-and-managing-large-language-models-us/3910996>,

- [30] L. Priya, Mastering LLMops: Best practices for managing and deploying large language models, 2024, URL: <https://lekha-bhan88.medium.com/mastering-llmops-best-practices-for-managing-and-deploying-large-language-models-c8ca0da648d9>,
- [31] T. Mboweni, T. Masombuka, C. Dongmo, A systematic review of machine learning DevOps, in: 2022 International Conference on Electrical, Computer and Energy Technologies, ICECET, IEEE, 2022, pp. 1–6, <http://dx.doi.org/10.1109/ICECET55527.2022.9872968>.
- [32] A. Lima, L. Monteiro, A.P. Furtado, MLOps: Practices, maturity models, roles, tools, and challenges-A systematic literature review, ICEIS (1) (2022) 308–320, <http://dx.doi.org/10.5220/0010997300003179>.
- [33] M. Testi, M. Ballabio, E. Frontoni, G. Iannello, S. Moccia, P. Soda, G. Vessio, MLOps: A taxonomy and a methodology, IEEE Access 10 (2022) 63606–63618, <http://dx.doi.org/10.1109/ACCESS.2022.3181730>.
- [34] A.B. Kolltveit, J. Li, Operationalizing machine learning models: A systematic literature review, in: Proceedings of the 1st Workshop on Software Engineering for Responsible AI, 2022, pp. 1–8, <http://dx.doi.org/10.1145/3526073.352758>.
- [35] G. Recupito, F. Pecorelli, G. Catolino, S. Moreschini, D. Di Nucci, F. Palomba, D.A. Tamburri, A multivocal literature review of MLOps tools and features, in: 2022 48th Euromicro Conference on Software Engineering and Advanced Applications, SEAA, IEEE, 2022, pp. 84–91, <http://dx.doi.org/10.1109/SEAA56994.2022.00021>.
- [36] B.C. Vadde, V.B. Munagandla, DevOps in the age of machine learning: Bridging the gap between development and data science, Int. J. Mach. Learn. Res. Cybersecur. Artif. Intell. 15 (1) (2024) 530–544, <http://dx.doi.org/10.70705/ppp.doaj.2024.v03.i01.pp18-24>.
- [37] S.J. Warnett, E. Ntentos, U. Zdun, A model-driven, metrics-based approach to assessing support for quality aspects in MLOps system architectures, J. Syst. Softw. 220 (2025) 112257, <http://dx.doi.org/10.1016/j.jss.2024.112257>.
- [38] S.J. Warnett, U. Zdun, Architectural design decisions for the machine learning workflow, Computer 55 (3) (2022) 40–51, <http://dx.doi.org/10.1109/MC.2021.3134800>.
- [39] S.J. Warnett, U. Zdun, On the understandability of MLOps system architectures, IEEE Trans. Softw. Eng. (2024) 1–25, <http://dx.doi.org/10.1109/TSE.2024.3367488>.
- [40] I. Kumara, R. Arts, D. Di Nucci, W.J. Van Den Heuvel, D.A. Tamburri, Requirements and reference architecture for MLOps: Insights from industry, Authora Prepr. (2023) <http://dx.doi.org/10.36227/techrxiv.21397413.v1>.
- [41] T. Gergen Erdogan, H. Altunel, A.K. Tarhan, A process model for AI-enabled software development: A synthesis from validation studies in white literature, J. Softw.: Evol. Process. 37 (1) (2024) e2743, <http://dx.doi.org/10.1002/smr.2743>.
- [42] A.P. Woźniak, M. Milczarek, J. Woźniak, MLOps components, tools, process and Metrics-A systematic literature review, IEEE Access (2025).
- [43] D.J. Patil, Building Data Science Teams, O'Reilly Media, Inc., 2011.
- [44] M. Kim, T. Zimmermann, R. DeLine, A. Begel, Data scientists in software teams: State of the art and challenges, IEEE Trans. Softw. Eng. 44 (11) (2018) 1024–1038, <http://dx.doi.org/10.1109/TSE.2017.2754374>.
- [45] J.S. Saltz, N.W. Grady, The ambiguity of data science team roles and the need for a data science workforce framework, in: 2017 IEEE International Conference on Big Data (Big Data), 2017, pp. 2355–2361, <http://dx.doi.org/10.1109/BigData.2017.8258190>.
- [46] S. Keele, B. Kitchenham, et al., Guidelines for Performing Systematic Literature Reviews in Software Engineering, Technical Report, ver. 2.3 ebse technical report. ebse, 2007.
- [47] C. Wohlin, Guidelines for snowballing in systematic literature studies and a replication in software engineering, in: Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering, EASE '14, ACM, 2014, <http://dx.doi.org/10.1145/2601248.2601268>.
- [48] A. Ampatzoglou, S. Bibi, P. Avgeriou, M. Verbeek, A. Chatzigeorgiou, Identifying, categorizing and mitigating threats to validity in software engineering secondary studies, Inf. Softw. Technol. 106 (2019) 201–230, <http://dx.doi.org/10.1016/j.infsof.2018.10.006>.
- [49] S. Moreschini, D. Hästbacka, D. Taibi, MLOps pipeline development: The OSSARA use case, in: Proceedings of the 2023 International Conference on Research in Adaptive and Convergent Systems, RACS '23, Association for Computing Machinery, New York, NY, USA, 2023, <http://dx.doi.org/10.1145/3599957.3606211>.
- [50] N. Petrović, Model-Driven Approach to Blockchain-Enabled MLOps, 2022.
- [51] Y. Zhou, Y. Yu, B. Ding, Towards MLOps: A case study of ML pipeline platform, in: 2020 International Conference on Artificial Intelligence and Computer Engineering, ICAICE, IEEE, 2020, pp. 494–500.
- [52] R. Dautov, E.J. Husom, F. Gonidis, Towards MLOps in mobile development with a plug-in architecture for data analytics, in: 2022 6th International Conference on Computer, Software and Modeling, ICCSM, IEEE, 2022, pp. 22–27.
- [53] S. Mei, C. Liu, Q. Wang, H. Su, Model provenance management in MLOps Pipeline, in: 2022 the 8th International Conference on Computing and Data Engineering, 2022, pp. 45–50.
- [54] S. Roh, K.-M. Jeong, H.-Y. Cho, E.-N. Huh, An efficient microservices architecture for MLOps, in: 2023 Fourteenth International Conference on Ubiquitous and Future Networks, ICUFN, IEEE, 2023, pp. 652–654.
- [55] R.B. Bahaweres, A. Zulfikar, I. Hermadi, A.I. Suroso, Y. Arkeman, Docker and kubernetes pipeline for DevOps software defect prediction with MLOps approach, in: 2022 2nd International Seminar on Machine Learning, Optimization, and Data Science, ISMODE, IEEE, 2022, pp. 248–253.
- [56] T.Y. Fujii, V.T. Hayashi, R. Arakaki, W.V. Ruggiero, R. Bulla Jr., F.H. Hayashi, K.A. Khalil, A digital twin architecture model applied with MLOps techniques to improve short-term energy consumption prediction, Machines 10 (1) (2021) 23.
- [57] I. Pölöskei, MLOps approach in the cloud-native data pipeline design, Acta Tech. Jaurinensis 15 (1) (2022) 1–6.
- [58] C. Min, A. Mathur, U.G. Acer, A. Montanari, F. Kawsar, SensiX++: Bringing MLOps and Multi-tenant Model Serving to Sensory Edge Devices, 2023, 3617507, <http://dx.doi.org/10.1145/3617507>.
- [59] M. Słupczyński, R. Klamka, MILKI-PSY Cloud: MLOps-based Multimodal Sensor Stream Processing Pipeline for Learning Analytics in Psychomotor Education, 2022.
- [60] M.N. Chowdary, B. Sankeerth, C.K. Chowdary, M. Gupta, Accelerating the machine learning model deployment using MLOps, in: Journal of Physics: Conference Series, 2327, (1) IOP Publishing, 2022, 012027.
- [61] D.R. Niranjana, Jenkins pipelines: A novel approach to machine learning operations (mlops), in: 2022 International Conference on Edge Computing and Applications, ICECAA, IEEE, 2022, pp. 1292–1297.
- [62] G. Zárate, R. Miñón, J. Díaz-de Arcaya, A.I. Torre-Bastida, K2E: Building MLOps environments for governing data and models catalogues while tracking versions, in: 2022 IEEE 19th International Conference on Software Architecture Companion (ICSA-C), IEEE, 2022, pp. 206–209, <http://dx.doi.org/10.1109/ICSA-C54293.2022.00047>.
- [63] M. Zeller, T. Waschulzik, R. Schmid, C. Bahlmann, Toward a safe MLOps process for the continuous development and safety assurance of ML-based systems in the railway domain, AI Ethics 4 (1) (2024) 123–130.
- [64] J.-h. Cha, H.-g. Jeong, S.-w. Han, D.-c. Kim, J.-h. Oh, S.-h. Hwang, B.-j. Park, Development of MLOps platform based on power source analysis for considering manufacturing environment changes in real-time processes, in: International Conference on Human-Computer Interaction, Springer, 2023, pp. 224–236.
- [65] Y. Ei, Speeding up the machine-learning process with MLOps and creating a mechanism to continuously provide service value, NTT Tech. Rev. 19 (5) (2021) 61–67.
- [66] S. Moreschi, G. Recupito, V. Lenarduzzi, F. Palomba, D. Hastbacka, D. Taibi, Toward end-to-end MLOps tools map: A preliminary study based on a multivocal literature review, 2023, arXiv preprint [arXiv:2304.03254](https://arxiv.org/abs/2304.03254). <http://dx.doi.org/10.48550/arXiv.2304.03254>.
- [67] E. Raj, D. Buffoni, M. Westerlund, K. Ahola, Edge MLOps: An automation framework for AIoT applications, in: 2021 IEEE International Conference on Cloud Engineering (IC2E), 2021, pp. 191–200, <http://dx.doi.org/10.1109/IC2E52221.2021.00034>.
- [68] P. Ruf, M. Madan, C. Reich, D. Ould-Abdeslam, Demystifying MLOps and presenting a recipe for the selection of open-source tools, Appl. Sci. 11 (1919) (2021) 8861, <http://dx.doi.org/10.3390/app1198861>.
- [69] T. Raffin, T. Reichenstein, J. Werner, A. Kühl, J. Franke, A reference architecture for the operationalization of machine learning models in manufacturing, Procedia CIRP 115 (2022) 130–135, <http://dx.doi.org/10.1016/j.procir.2022.10.062>.
- [70] I. Karamitsos, S. Albarhami, C. Apostolopoulos, Applying DevOps practices of continuous automation for machine learning, Information 11 (77) (2020) 363, <http://dx.doi.org/10.3390/info11070363>.
- [71] Y. Liu, Z. Ling, B. Huo, B. Wang, T. Chen, E. Mouine, Building a platform for machine learning operations from open source frameworks, in: 3rd IFAC Workshop on Cyber-Physical & Human Systems CPHS 2020, IFAC-PapersOnLine 53 (5) (2020) 704–709, <http://dx.doi.org/10.1016/j.ifacol.2021.04.161>.
- [72] M. Antonini, M. Pincheira, M. Vecchio, F. Antonelli, Tiny-MLOps: a framework for orchestrating ML applications at the far edge of IoT systems, in: 2022 IEEE International Conference on Evolving and Adaptive Intelligent Systems, EAIS, 2022, pp. 1–8, <http://dx.doi.org/10.1109/EAIS51927.2022.9787703>.
- [73] V. Janapa Reddi, A. Elum, S. Hymel, D. Tischler, D. Situnayake, C. Ward, L. Moreau, J. Plunkett, M. Kelcey, M. Baajenis, et al., Edge impulse: An MLOps platform for tiny machine learning, Proc. Mach. Learn. Syst. 5 (2023) 254–268.
- [74] A. Banerjee, C.-C. Chen, C.-C. Hung, X. Huang, Y. Wang, R. Chevesaran, Challenges and experiences with MLOps for performance diagnostics in hybrid-cloud enterprise software deployments, in: 2020 USENIX Conference on Operational Machine Learning (OpML 20), 2020.
- [75] M. Barry, J. Montiel, A. Bifet, S. Wadkar, N. Manchev, M. Halford, R. Chiky, S.E. Jaouhari, K.B. Shakman, J.A. Fehailly, F. Le Deit, V.-T. Tran, E. Guerizec, StreamMLOps: Operationalizing online learning for big data streaming and real-time applications, in: 2023 IEEE 39th International Conference on Data Engineering, ICDE, 2023, pp. 3508–3521, <http://dx.doi.org/10.1109/ICDE55515.2023.00272>.
- [76] Y. Akkem, S.K. Biswas, A. Varanasi, Smart farming monitoring using ML and MLOps, in: A.E. Hassanien, O. Castillo, S. Anand, A. Jaiswal (Eds.), International Conference on Innovative Computing and Communications, in: Lecture Notes in Networks and Systems, Springer Nature, Singapore, 2023, pp. 665–675, http://dx.doi.org/10.1007/978-981-99-3315-0_51.

- [77] Y. Luo, M. Raatikainen, J.K. Nurminen, Autonomously adaptive machine learning systems: Experimentation-driven open-source pipeline, in: 2023 49th Euromicro Conference on Software Engineering and Advanced Applications, SEAA, 2023, pp. 44–52, <http://dx.doi.org/10.1109/SEAA60479.2023.00016>.
- [78] S. Kwatra, U. Mangla, M. Srivatsa, Model operationalization at edge devices, in: M. Srivatsa, T. Abdelzaher, T. He (Eds.), Artificial Intelligence for Edge Computing, Springer International Publishing, Cham, 2023, pp. 353–365, http://dx.doi.org/10.1007/978-3-031-40787-1_12.
- [79] A.Y. Sitcheu, N. Friederich, S. Baeuerle, O. Neumann, M. Reischl, R. Mikut, MLOps for Scarce Image Data: A Use Case in Microscopic Image Analysis, 2023, [arXiv:2309.15521](https://arxiv.org/abs/2309.15521), <http://dx.doi.org/10.48550/arXiv.2309.15521>.
- [80] R. Semola, V. Lomonaco, D. Bacciu, Continual-learning-as-a-service (claas): On-demand efficient adaptation of predictive models, 2022, [arXiv:2206.06957](https://arxiv.org/abs/2206.06957).
- [81] X. Zhang, J. Jaskolka, Conceptualizing the secure machine learning operations (SecMLOps) paradigm, in: 2022 IEEE 22nd International Conference on Software Quality, Reliability and Security, QRS, 2022, pp. 127–138, <http://dx.doi.org/10.1109/QRS57517.2022.00023>.
- [82] M. Safdar, P.P. Paul, G. Lamouche, G. Wood, M. Zimmermann, F. Hannesen, C. Bescond, P. Wanjara, Y.F. Zhao, Fundamental requirements of a machine learning operations platform for industrial metal additive manufacturing, *Comput. Ind.* 154 (2024) 104037, <http://dx.doi.org/10.1016/j.compind.2023.104037>.
- [83] L. Faubel, K. Schmid, H. Eichelberger, Is MLOps different in industry 4.0? General and specific challenges, in: Proceedings of the 3rd International Conference on Innovative Intelligent Industrial Production and Logistics, SCITEPRESS - Science and Technology Publications, Valletta, Malta, 2022, pp. 161–167, <http://dx.doi.org/10.5220/0011589600003329>.
- [84] C. Heged s, P. Varga, Tailoring MLOps techniques for industry 5.0 needs, in: 2023 19th International Conference on Network and Service Management, CNSM, 2023, pp. 1–7, <http://dx.doi.org/10.23919/CNSM59352.2023.10327814>.
- [85] M.Y.S. Krishna, S.K. Gawre, MLOps for enhancing the accuracy of machine learning models using DevOps, continuous integration, and continuous deployment, *Res. Rep. Comput. Sci.* (2023) 97–103, <http://dx.doi.org/10.37256/rres.2320232644>.
- [86] P. Ruf, C. Reich, D. Ould-Abdeslam, Aspects of module placement in machine learning operations for cyber physical systems, in: 2022 11th Mediterranean Conference on Embedded Computing, MECO, 2022, pp. 1–6, <http://dx.doi.org/10.1109/MECO55406.2022.9797080>.
- [87] Y. Zhao, A.S.Z. Belloum, G.M.d. Costa, Z. Zhao, MLOps scaling machine learning lifecycle in an industrial setting, *Int. J. Ind. Manuf. Eng.* 16 (5) (2022) 138–148.
- [88] V.-H. Pham, H.K. Nghi, H.Q. Nguyen, D.T. Phan, Deception and continuous training approach for web attack detection using cyber traps and MLOps, *VNUHCM J. Sci. Technol. Dev.* 26 (2) (2023) 2729–2740, <http://dx.doi.org/10.32508/stdj.v26i2.4044>.
- [89] E. Peltonen, S. Dias, LinkEdge: Open-sourced MLOps Integration with IoT Edge, in: Proceedings of the 3rd Eclipse Security, AI, Architecture and Modelling Conference on Cloud To Edge Continuum, ESAAM '23, Association for Computing Machinery, New York, NY, USA, 2023, pp. 67–76, <http://dx.doi.org/10.1145/3624486.3624496>.
- [90] Z.S. Elgamal, L. Elfangary, H. Fahmy, A machine learning operations (MLOps) monitoring model using BI-LSTM and SARSA algorithms, *Int. J. Adv. Comput. Sci. Appl.* 15 (10) (2024) <http://dx.doi.org/10.14569/IJACSA.2024.0151060>, URL: <http://thesai.org/Publications/ViewPaper?Volume=15&Issue=10&Code=ijacsa&SerialNo=60>.
- [91] O.A. M ndez, J. Camargo, H. Florez, Machine learning operations applied to development and model provisioning, in: H. Florez, H. Astudillo (Eds.), Applied Informatics, in: Communications in Computer and Information Science, vol. 2236, Springer Nature Switzerland, Cham, 2025, pp. 73–88, http://dx.doi.org/10.1007/978-3-031-75144-8_6, Series Title.
- [92] C.-C. Hsu, P.-H. Chen, I.-Z. Wu, End-to-end automation of ML model lifecycle management using machine learning operations platforms, in: 2024 International Conference on Consumer Electronics - Taiwan (ICCE-Taiwan), IEEE, Taichung, Taiwan, 2024, pp. 209–210, <http://dx.doi.org/10.1109/ICCE-Taiwan62264.2024.10674445>.
- [93] A.H. van Bruggen, K. Kruger, A.H. Basson, J. Grobler, An architecture to integrate digital twins and machine learning operations, in: T. Borangiu, D. Trentesaux, P. Leit o, L. Berrah, J.-F. Jim nez (Eds.), Service Oriented, Holonic and Multi-Agent Manufacturing Systems for Industry of the Future, Springer Nature Switzerland, Cham, 2024, pp. 3–14.
- [94] J. Conde, A. Munoz-Arcenales, A. Alonso, J. Salvach a, G. Huecas, Enhanced FIWARE-based architecture for cyberphysical systems with tiny machine learning and machine learning operations: A case study on urban mobility systems, *IT Prof.* 26 (5) (2024) 55–61, <http://dx.doi.org/10.1109/MITP.2024.3421968>.
- [95] A.M. Burgue o-Romero, C. Barba-Gonz lez, J.F. Aldana-Montes, Big Data-driven MLOps workflow for annual high-resolution land cover classification models, *Future Gener. Comput. Syst.* 163 (2025) 107499, <http://dx.doi.org/10.1016/j.future.2024.107499>.
- [96] N. Sirisha, A. Kiran, M. Arshad, M. M., Automating ML models using MLOps, in: 2024 International Conference on Advancements in Smart, Secure and Intelligent Computing (ASSIC), IEEE, Bhubaneswar, India, 2024, pp. 1–5, <http://dx.doi.org/10.1109/ASSIC60049.2024.10507923>.
- [97] H. Safri, G. Papadimitriou, E. Deelman, Dynamic tracking, MLOps, and workflow integration: Enabling transparent reproducibility in machine learning, in: 2024 IEEE 20th International Conference on E-Science (E-Science), IEEE, Osaka, Japan, 2024, pp. 1–10, <http://dx.doi.org/10.1109/e-Science62913.2024.10678658>.
- [98] G. Mallardi, F. Calefato, L. Quaranta, F. Lanubile, An MLOps approach for deploying machine learning models in healthcare systems, in: 2024 IEEE International Conference on Bioinformatics and Biomedicine (BIBM), IEEE, Lisbon, Portugal, 2024, pp. 6832–6837, <http://dx.doi.org/10.1109/BIBM62325.2024.10822603>.
- [99] A.C. Cob-Parro, Y. Lalangui, R. Lazcano, Fostering agricultural transformation through AI: An open-source AI architecture exploiting the MLOps paradigm, *Agronomy* 14 (2) (2024) 259, <http://dx.doi.org/10.3390/agronomy14020259>.
- [100] L. Faubel, T. Woudsma, L. Methnani, A.G. Ghezljhemaidan, F. Buelow, K. Schmid, W.D. Van Driel, B. Kloepper, A. Theodorou, M. Nosratinia, M. B  ang, A MLOps architecture for XAI in industrial applications, in: 2024 IEEE 29th International Conference on Emerging Technologies and Factory Automation (ETFA), IEEE, Padova, Italy, 2024, pp. 1–4, <http://dx.doi.org/10.1109/ETFA61755.2024.10711084>.
- [101] A. Bodor, M. Hnida, N. Daoudi, Machine learning models monitoring in MLOps context: Metrics and tools, *Int. J. Interact. Mob. Technol. (IJIM)* 17 (23) (2023) 125–139, <http://dx.doi.org/10.3991/ijim.v17i23.43479>.
- [102] V. Kumar, D. Ghosh, S. Srivastava, Efficient MLOps pipeline for transfer learning and reuse of pre-trained ML models, in: 2023 IEEE International Conference on Advanced Networks and Telecommunications Systems (ANTS), 2023, pp. 1–6, <http://dx.doi.org/10.1109/ANTS59832.2023.10468932>.
- [103] A.K. Jana, Framework for automated machine learning workflows: Building end-to-end MLOps tools for scalable systems on AWS, *J. Artif. Intell. Mach. Learn. Data Sci.* 1 (3) (2023) 575–579, <http://dx.doi.org/10.51219/JAIMLD/aryyama-kumar-jana/151>.
- [104] H. Bhatt, S. Arun, A. Kakran, K. Vaidhyanathan, Towards architecting sustainable MLOps: A self-adaptation approach, in: 2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C), IEEE, Hyderabad, India, 2024, pp. 179–182, <http://dx.doi.org/10.1109/ICSA-C63560.2024.00038>.
- [105] A.M. Burgue o-Romero, A. Ben tez-Hidalgo, C. Barba-Gonz lez, J.F. Aldana-Montes, Toward an open source MLOps architecture, *IEEE Softw.* 42 (1) (2025) 59–64, <http://dx.doi.org/10.1109/MS.2024.3421675>.
- [106] A. Bodor, M. Hnida, D. Najima, From development to deployment: An approach to MLOps monitoring for machine learning model operationalization, in: 2023 14th International Conference on Intelligent Systems: Theories and Applications (SITA), IEEE, Casablanca, Morocco, 2023, pp. 1–7, <http://dx.doi.org/10.1109/SITA60746.2023.10373733>.
- [107] A.H. Nia, F.J. Kaleibar, F. Feizi, F. Rahimi, H. Kashfi, Unlocking the power of data in telecom: Building an effective MLOps infrastructure for model deployment, in: 2023 7th Iranian Conference on Advances in Enterprise Architecture (ICA EA), IEEE, Tehran, Iran, Islamic Republic of, 2023, pp. 78–84, <http://dx.doi.org/10.1109/ICA EA60387.2023.10414445>.
- [108] Abhishek, R. Tripathi, Automated credit card default prediction using MLOps on AWS: From pipeline development to real-time deployment, in: 2024 First International Conference on Technological Innovations and Advance Computing (TIACOMP), IEEE, Bali, Indonesia, 2024, pp. 250–255, <http://dx.doi.org/10.1109/TIACOMP64125.2024.00050>.
- [109] F.J. Iriarte, M.E. Ortiz, L. Unzueta, J. Mart nez, J. Zaldivar, I. Arganda-Carreras, Multi-level XAI-driven MLOps pipeline for the adjustment of fruit and vegetable classifiers, in: 2024 IEEE 12th International Conference on Intelligent Systems (IS), IEEE, Varna, Bulgaria, 2024, pp. 1–6, <http://dx.doi.org/10.1109/IS61756.2024.10705202>.
- [110] D.C. Da Silva, M.A.F. De Sousa, W. De Oliveira, A. Barbosa, G. Bressan, R.M. Silveira, P.S.R. Henrique, R. Prasad, NSSF function in 6G networks based on MLOps deployment model, in: 2024 27th International Symposium on Wireless Personal Multimedia Communications (WPMC), IEEE, Greater Noida, India, 2024, pp. 1–6, <http://dx.doi.org/10.1109/WPMC63271.2024.10863370>.
- [111] N. Peyrone, D. Wichadakul, A formal model for integrating consent management into MLOps, *IEEE Access* 12 (2024) 142524–142541, <http://dx.doi.org/10.1109/ACCESS.2024.3471773>.
- [112] N. Shakhovska, J. Campos, Predictive maintenance for wind turbine bearings: An MLOps approach with the DIAFS machine learning model, in: SMARTINDUSTRY-2024: International Conference on Smart Automation & Robotics for Future Industry, April 18–20, 2024, Lviv, Ukraine, 2024.
- [113] A. Bodor, M. Hnida, N. Daoudi, Integration of web scraping, fine-tuning, and data enrichment in a continuous monitoring context via large language model operations, *Int. J. Electr. Comput. Eng. (IJECE)* 15 (1) (2025) 1027, <http://dx.doi.org/10.11591/ijece.v15i1.pp1027-1037>.

- [114] A. Jain, A. Malviya, D. Bajaj, R. Bhavsar, A. Savyanavar, Brain tumor detection using MLOps and hybrid multi-cloud, in: 2022 IEEE International Conference on Blockchain and Distributed Systems Security, ICBDS, 2022, pp. 1–6, <http://dx.doi.org/10.1109/ICBDS53701.2022.9936020>.
- [115] S.K. Paul, S. Riaz, S. Das, A conceptual architecture for AI in supply chain risk management, in: TENCON 2022 - 2022 IEEE Region 10 Conference, TENCON, 2022, pp. 1–5, <http://dx.doi.org/10.1109/TENCON55691.2022.9978154>.
- [116] R. Ranawana, A.S. Karunananda, An Agile software development life cycle model for machine learning application development, in: 2021 5th SLAAI International Conference on Artificial Intelligence (SLAAI-ICAI), 2021, pp. 1–6, <http://dx.doi.org/10.1109/SLAAI-ICAI54477.2021.9664736>.
- [117] J. Pineda-Jaramillo, F. Viti, MLOps in freight rail operations, Eng. Appl. Artif. Intell. 123 (2023) 106222, <http://dx.doi.org/10.1016/j.engappai.2023.106222>.
- [118] T. Granlund, V. Stirbu, T. Mikkonen, Towards regulatory-compliant MLOps: Oravizio's journey from a machine learning experiment to a deployed certified medical product, SN Comput. Sci. 2 (5) (2021) 342, <http://dx.doi.org/10.1007/s42979-021-00726-1>.
- [119] T.-A. Tran, T. Ruppert, J. Abonyi, The use of explainable artificial intelligence and machine learning operation principles to support the continuous development of machine learning-based solutions in fault detection and identification, Computers 13 (10) (2024) 252, <http://dx.doi.org/10.3390/computers13100252>.
- [120] A. Gopikrishnan, A. Rao, A. Prakash, V. Gowtham, F. Schreiner, M. Corici, C. Hein, T. Magedanz, Machine learning pipeline for anomaly detection in next generation networks, in: 2024 IEEE Conference on Standards for Communications and Networking, CSCN, 2024, pp. 196–201, <http://dx.doi.org/10.1109/CSCN63874.2024.10849724>.
- [121] Y. Seo, H.-k. Jo, S. Roh, T.-J. Sun, S.-W. Jeong, H. Kim, T.-T. Ngo, E.-N. Huh, DevoX: Deep voice detection MLOps framework for crime prevention in the core-edge cloud, in: 2024 International Conference on Information Networking, ICOIN, 2024, pp. 678–682, <http://dx.doi.org/10.1109/ICOIN59985.2024.10572202>.
- [122] H. Kim, B. Kim, W. Lu, L. Li, No-code MLOps platform for data annotation, in: 2023 IEEE International Conference on Memristive Computing and Applications (ICMCA), 2023, pp. 1–6, <http://dx.doi.org/10.1109/ICMCA59770.2023.10480992>.
- [123] L. Faubel, K. Schmid, MLOps: A multiple case study in industry 4.0, in: 2024 IEEE 29th International Conference on Emerging Technologies and Factory Automation (ETFA), 2024, pp. 01–08, <http://dx.doi.org/10.1109/ETFA61755.2024.10711136>.
- [124] Y. Billeter, P. Denzel, R. Chavarriaga, O. Forster, F.-P. Schilling, S. Brunner, C. Frischknecht-Gruber, M. Reif, J. Weng, MLOps as enabler of trustworthy AI, in: 2024 11th IEEE Swiss Conference on Data Science, SDS, 2024, pp. 37–40, <http://dx.doi.org/10.1109/SDS60720.2024.00013>.
- [125] A.L. Ferreira, J.M. Fernandes, MLOps for developing machine-learning-enhanced automotive applications: Experience at the Bosch cross-domain computing solutions division, IEEE Softw. 42 (1) (2025) 34–41, <http://dx.doi.org/10.1109/MS.2024.3415750>.
- [126] L. Colombi, A. Gilli, S. Dahdal, I. Boleac, M. Tortonesi, C. Stefanelli, M. Vignoli, A machine learning operations platform for streamlined model serving in industry 5.0, in: NOMS 2024-2024 IEEE Network Operations and Management Symposium, 2024, pp. 1–6, <http://dx.doi.org/10.1109/NOMS59830.2024.10575103>.
- [127] A. Chatterjee, B.S. Ahmed, E. Hallin, A. Engman, Quality assurance in MLOps setting: An industrial perspective, 2022, arXiv preprint [arXiv:2211.12706](https://arxiv.org/abs/2211.12706).
- [128] A.P. Gonzalez Torres, N. Sawhney, Role of regulatory sandboxes and MLOps for AI-enabled public sector services, Rev. Socionetwork Strat. 17 (2) (2023) 297–318, <http://dx.doi.org/10.1007/s12626-023-00146-y>.
- [129] O. Spjuth, F. Jens, A. Hellander, The machine learning life cycle and the cloud: implications for drug discovery, Expert. Opin. Drug Discov. 16 (9) (2021) 1071–1079, <http://dx.doi.org/10.1080/17460441.2021.1932812>.
- [130] C. Wu, E. Haihong, M. Song, An automatic artificial intelligence training platform based on kubernetes, in: Proceedings of the 2020 2nd International Conference on Big Data Engineering and Technology, 2020, pp. 58–62.
- [131] B. Karlaš, M. Interlandi, C. Renggli, W. Wu, C. Zhang, D. Mukunthu Iyapan Babu, J. Edwards, C. Lauren, A. Xu, M. Weimer, Building continuous integration services for machine learning, in: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD '20, Association for Computing Machinery, New York, NY, USA, 2020, pp. 2407–2415, <http://dx.doi.org/10.1145/3394486.3403290>.
- [132] M.G. Usha, M.S. Shreya, S. Supreeth, G. Shruthi, D. Pruthviraja, P. Chavan, Kidney tumor detection using MLflow, DVC and deep learning, in: 2024 Second International Conference on Advances in Information Technology (ICAIT), vol. 1, 2024, pp. 1–7, <http://dx.doi.org/10.1109/ICAIT61638.2024.10690537>.
- [133] R.J. Priyankasingh, H.Y. Vani, Deployment and serving ML models using kubeflow and kfserving, in: A.B. Reddy, S. Nagini, V.E. Balas, K.S. Raju (Eds.), Proceedings of Third International Conference on Advances in Computer Engineering and Communication Systems, Springer Nature Singapore, Singapore, 2023, pp. 283–293.
- [134] A. Saputra, E. Suryani, N.A. Rakhmawati, Secure and scalable LLM-based recommendation systems: An MLOps and security by design, in: 2024 IEEE International Symposium on Consumer Technology (ISCT), 2024, pp. 623–629, <http://dx.doi.org/10.1109/ISCT62336.2024.10791207>.
- [135] E.M.-Y. Tan, F. Liu, Y.K. Chia, A framework for adopting machine learning in the clinical domain, in: Data, Information and Computing Science, IOS Press, 2025, pp. 78–87, <http://dx.doi.org/10.3233/ATDE250010>, URL: <https://ebooks.iospress.nl/doi/10.3233/ATDE250010>.
- [136] N. Siilasjoki, N. Raatikainen, J.K. Nurminen, A design framework for cloud-HPC integration in MLOps, 2025.
- [137] R. Subramanya, P. Räisänen, S. Sierla, V. Vyatkin, Cloud computing design patterns for MLOps: Applications to virtual power plants, in: IECON 2023- 49th Annual Conference of the IEEE Industrial Electronics Society, 2023, pp. 01–07, <http://dx.doi.org/10.1109/IECON51785.2023.10312212>.
- [138] T. Zimmermann, Card-sorting: From text to themes, in: Perspectives on Data Science for Software Engineering, Elsevier, 2016, pp. 137–141, <http://dx.doi.org/10.1016/B978-0-12-804206-9.00027-1>.
- [139] W. Hudson, Card sorting, *Encycl. Human-Computer Interact.* (2014) Accessed Mar. 31, 2024.
- [140] S. Martínez-Fernández, J. Bogner, X. Franch, M. Oriol, J. Siebert, A. Trendowicz, A.M. Vollmer, S. Wagner, Software engineering for AI-based systems: A survey, ACM Trans. Softw. Eng. Methodol. 31 (2) (2022) <http://dx.doi.org/10.1145/3487043>.
- [141] F. Amou Najafabadi, J. Bogner, I. Gerostathopoulos, P. Lago, An architectural perspective on MLOps: Structures, processes, tools, and stakeholders, Zenodo, 2026. <http://dx.doi.org/10.5281/zenodo.16787882>.
- [142] K.H. Russ Miles, *Learning UML 2.0*, O'Reilly Media, Inc, 2006.
- [143] F. Modeling, The fundamental modeling concepts, 2014, <http://www.fmc-modeling.org>. (Accessed 31 March 2024).
- [144] K. Salama, J. Kazmierczak, D. Schut, Practitioners guide to MLOps: A framework for continuous delivery and automation of machine learning, 2021, URL: https://services.google.com/fh/files/misc/practitioners_guide_to_mlops_whitepaper.pdf. (Accessed 1 December 2025).
- [145] J. Kazmierczak, K. Salama, V. Huerta, MLOps: Continuous delivery and automation pipelines in machine learning, 2024, URL: <https://docs.cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning>.
- [146] Microsoft, MLOps maturity model, 2025, URL: <https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/mlops-maturity-model>.
- [147] J. Bradley, R. Kurlansik, M. Thomson, N. Turbitt, The Big Book of MLOps, 2025, URL: <https://www.databricks.com/resources/ebook/the-big-book-of-mlops>. (Accessed 1 December 2025).
- [148] F. Bayram, B.S. Ahmed, Towards trustworthy machine learning in production: An overview of the robustness in MLOps approach, ACM Comput. Surv. 57 (5) (2025) 1–35.
- [149] A. Corso, D. Karamadian, R. Valentin, M. Cooper, M.J. Kochenderfer, A holistic assessment of the reliability of machine learning systems, 2023, arXiv preprint [arXiv:2307.10586](https://arxiv.org/abs/2307.10586).
- [150] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al., Training language models to follow instructions with human feedback, Adv. Neural Inf. Process. Syst. 35 (2022) 27730–27744.
- [151] C. Wohlin, P. Runeson, M. Höst, M.C. Ohlsson, B. Regnell, A. Wesslén, Experimentation in Software Engineering, Springer Science & Business Media, 2012, <http://dx.doi.org/10.1007/978-3-662-69306-3>.
- [152] P. Runeson, M. Höst, Guidelines for conducting and reporting case study research in software engineering, Empir. Softw. Eng. 14 (2009) 131–164, <http://dx.doi.org/10.1007/s10664-008-9102-8>.